

Received 5 November 2024, accepted 21 November 2024, date of publication 25 November 2024, date of current version 4 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3505546

RESEARCH ARTICLE

A Hybrid Task Scheduling Technique in Fog Computing Using Fuzzy Logic and Deep Reinforcement Learning

PRASHANTH CHOPPARA¹ AND S. SUDHEER MANGALAMPALLI², (Member, IEEE)

¹School of Computer Science and Engineering, VIT-AP University, Amaravati, Andhra Pradesh 522237, India

²Department of Computer Science and Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, Karnataka 576104, India

Corresponding author: S. Sudheer Mangalampalli (ms.sudheer@manipal.edu)

This work was supported by the Manipal Academy of Higher Education.

ABSTRACT This work presents an innovative method for scheduling tasks in a fog computing environments by combining the fuzzy logic with deep reinforcement learning. In Internet of Things there has been a significant raising the amount of data produced by different devices. This has created a need for more effective methods of processing and managing this data. Conventional cloud computing often fails to fulfill the need of IoT usage in terms of high bandwidth, low makespan, and real-time processing. Fog computing presents available solution by placing the processing resources near the data source but the issue of efficient task scheduling remains a major obstacle. We proposed a technique that combines an Hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning (HTSFFDRL) algorithm with a Takagi-Sugeno fuzzy inference system. By continuously interacting with the environment, this hybrid technique allows for the dynamic prioritization of tasks and the real-time change of scheduling rules. The technique seeks to maximize many crucial performance measures, such as makespan, energy consumption, cost, and fault tolerance. Simulations extensively validate the suggested strategy, demonstrating significant enhancements compared to current approaches like LSTM, DQN, and A2C. The results indicate that combining fuzzy logic with reinforcement learning may greatly improve the effectiveness and dependability of task scheduling in fog computing, opening up possibilities for more resilient IoT applications.

INDEX TERMS Task scheduling, reinforcement learning, fuzzy, fog computing, makespan, cost, energy consumption, fault tolerance.

I. INTRODUCTION

The active development of Internet of Things (IoT) technology has significantly influenced several aspects of everyday life, introducing multiple opportunities for technical progress. The IoT industry is vast and rapidly growing, with a major increase in the importance and impact of IoT-enabled devices in the digital [1]. These gadgets include a wide range of products, including smart wearables like smartwatches and smart glasses, healthcare monitoring systems for metrics such as heart rate and glucose levels as well as smart home technologies and intelligent cars [2]. Industrial automation

systems play an important role in this ecosystem because they create and send large amounts of data among millions of units worldwide. Internet of MultiMedia Things (IoMT) stands out among the other subsystems in the IoT architecture because of its need for high network bandwidth, storage capacity, compute power. A network design that ensures prompt and dependable data transmission is essential for properly supporting IoMT applications. An account of multimedia data generated by IoMT applications frequently exceeds the capabilities of current protocols for real-time data transport and streaming necessitating either modifications to existing network structures or the creation of new systems [3], [4]. In this case cloud computing is critical because it provides virtualized resources that make it easier to transport and

The associate editor coordinating the review of this manuscript and approving it for publication was Hadi Tabatabaee Malazi¹.

manage data from IoT devices. However, the combination of IoT with cloud computing brings up other obstacles, such as the need for effective data transmission, storage, network administration, security concerns. IoT platforms progress in next-generation wireless networks, future communication standards must incorporate quality of service and location awareness, real-time data transmission, and latency sensitivity [5]. Fog computing emerge as a feasible solution to these problems placing computer resources in closer proximity to IoT devices. Unlike traditional centralized cloud computing, fog computing operates at the network's edge, addressing the required for high bandwidth, low latency, and real-time data processing. In the context of the IoT workflows are often represented as directed acyclic graphs (DAGs) that include several computing activities [6]. In fog computing settings, efficient workflow scheduling necessitates the intelligent assignment of activities to virtual resources according to their execution sequence with the goal of fulfilment load balance, energy efficiency and optimum resource management. Task scheduling becomes more complicated as the number of users increases, making it a challenging optimization issue that falls below the grouping of NP-hard problems.

This technique focuses on many crucial performance criteria, such as makespan, energy usage, cost, and fault tolerance. Efficient scheduling involves strategically determining the timing and distribution of tasks and resources in order to minimize delays and maximize throughput [7]. This leads to a reduced makespan and enhanced overall system efficiency. Inadequate scheduling may lead to suboptimal use of resources, resulting in a rise in energy usage. Alternatively, scheduling systems that take into account the energy needs of tasks and resources may eliminate unnecessary computational load, leading to decreased energy use, cost reduction, and environmental sustainability. Efficient time management is essential for maintaining cost control [8]. Insufficient scheduling may result in improper distribution of resources and increased operational costs due to inefficient resource use and heightened energy consumption. To ensure cost-efficient resource use and effective management of operational expenditures, efficient scheduling requires striking a harmonious equilibrium between resource needs and budget constraints. The effectiveness of fault tolerance also depends on the quality of work scheduling [9], [10].

Fog computing is a essential framework to deal with the growing requirements of Internet of Things applications, especially in terms of task scheduling and resource allocation. As with the growing volume and complexity of IoT tasks, proper management of these tasks becomes critical to maintain the best possible system performance. The conventional scheduling techniques fail to address the complexities of dynamic resource availability and the real-time demand nature of IoT workloads. Contending models like DQN and A2C can significantly improve task management, yet realize

a too significant shortage of adaptability and multi-objective optimization capabilities. To overcome the above challenges, this paper proposes Hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning (HTSFFDRL) for effective task scheduling in fog computing environments.

A. MOTIVATION AND KEY CHALLENGES

Rapid advancement of IoT technology has a new age of interconnection and automation, fundamentally transforming several aspects of daily life. It includes a diverse array of gadgets, including intelligent wearable, healthcare monitoring systems, and industrial automation tools. Efficient processing and administration are necessary for handling substantial amount of data was generated by IoT devices. The data has emphasized the need for advanced computing models capable of handling the rapid data transmission rate, low latency, real-time processing demands of IoT applications. Fog computing environments are dynamic and difficult to schedule resource-intensive tasks. Traditional methods are ineffective in meeting real-time requirements, which severely restricts efficient task allocation an indispensable requirement in time-sensitive IoT applications. As the number and complexity of IoT tasks increase, reducing makespan and saving energy are essential for the sustainable operation of fog computing environments. However, many current approaches fail to achieve such a balance, resulting in high costs and low reliability. The proposed work HTSFFDRL and A Takagi-Sugeno fuzzy system is applied in the framework to perform tasks dynamically based on live CPU utilization and memory requirements in HTSFFDRL. Optimizing an actor-critical reinforcement learning framework becomes a makespan-optimized task scheduling framework, while a multi-objective reward function controls makespan, energy consumption, cost, and fault tolerance for adaptive task scheduling.

B. CONTRIBUTIONS

This paper presents the several significant contributions to the field of task scheduling in fog computing

- We proposed a hybrid method that leverages the strengths of fuzzy logic and deep reinforcement learning to address the uncertainties and dynamic challenges of task scheduling in fog computing. Fuzzy logic component allows for the flexible prioritization of tasks based on multiple parameters. The deep reinforcement learning component (improved HTSFFDRL) enables the dynamic adjustment of scheduling policies through continuous interaction with the environment.
- This paper introduces a Takagi-Sugeno fuzzy inference system for task prioritization. This system offers faster processing, computational efficiency, accuracy, and adaptability compared to traditional fuzzy logic methods. It effectively handles the complexity and vari-

ability of task attributes, enhancing the decision-making process in task scheduling.

- A detailed reward function is formulated to guide the reinforcement learning process. The reward function incorporates multiple objectives task completion time, resource utilization, energy efficiency, cost, fault tolerance. It ensures that the scheduling policies developed by the HTSFFDRL algorithm optimize overall system performance and reliability.
- The paper presents the implementation of the improved HTSFFDRL algorithm tailored for fog computing paradigm. Extensive simulations demonstrate the algorithm's effectiveness in improving key performance metrics named makespan, energy consumption, cost, fault tolerance, using various datasets.
- The proposed approach is thoroughly evaluated against existing task scheduling methods are LSTM, DQN, A2C. The evaluation showcases significant improvements in scheduling efficiency and system performance. The results highlight the potential hybrid fuzzy logic, reinforcement learning approach to advance the state-of-the-art in task scheduling for fog computing.

Rest of the manuscript is organized as follows. Related works discussed in Section II, System model discussed in Section III, Mathematical modeling & proposed HTSFFDRL discussed in Sections IV and V, Section VI discusses experimental results. Section VII discusses conclusion & future works.

II. RELATED WORKS

In section II we discuss about the scheduling techniques for tasks in fog layer environment. In recent years various task scheduling models have been developed to enhance resource allocation in fog computing, which is vital for managing the growing number of IoT devices. The approaches in this area are Deep Q-Networks (DQN) and Advantage Actor-Critic (A2C). While these models show promise, they come with significant limitations that hinder their effectiveness in dynamic fog environments. One major issue is that they often operate under the assumption of stable resource availability, which makes it challenging for them to adapt to fluctuations in resource constraints. Additionally, these models typically focus on single-objective optimization, such as minimizing makespan or maximizing throughput, neglecting other crucial factors like energy efficiency, cost, and fault tolerance. This narrow focus can result in less effective scheduling outcomes in complex fog computing scenarios. Furthermore, their lack of real-time processing capabilities limits their usefulness in environments where task demands and resource availability can change rapidly. These shortcomings underscore the need for a more sophisticated approach to task scheduling. The Hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning (HTSFFDRL)

is proposed to address these issues by integrating fuzzy logic and reinforcement learning, allowing for adaptive task prioritization based on real-time assessments of system conditions. This comprehensive approach aims to improve task management in fog computing environments significantly.

In authors [11] developed a scheduling algorithm aimed at optimizing task allocation in fog computing. The primary objectives are to reduce both latency and energy consumption, which are critical for the efficiency and sustainability of IoT networks. The authors provide a reinforcement learning method that adaptive modifies task scheduling choices according to the present network conditions. The methodology entails representing task scheduling issue as a Markov Decision Process which uses Q-learning algorithm to ascertain most advantageous scheduling strategies. Extensive simulations confirm that the suggested technique surpasses conventional scheduling techniques by minimizing task latency, energy consumption. Hence improving overall network performance and sustainability.

In [12] author examined challenges of managing the large amount of data generated by intelligent medical devices in healthcare environments. Proposed system leverages advanced deep reinforcement learning techniques named genetic algorithm (GA), SARSA, Q-learning to enhance resource allocation and load balancing. The author proposed a novel method to improve the system throughput, makespan, and latency. conventional methods in terms of efficiency and effectiveness when implemented in MATLAB. In order to optimize the efficacy of healthcare systems, our research emphasizes the importance of load balancing. In addition, they identified potential areas of future research, such as utilization of multiple concurrent learning agents to improve scheduling efficiency.

The author of [13] discusses problem effectively organizing bag-of-tasks (BoT) applications in diverse fog computing environments. They proposed a novel multi-objective scheduling technique, with stated objective of minimizing both the cost of an activity and its execution time. The authors formulated this assignment as an optimization problem with multiple objectives due to variety of fog nodes and the dynamic pricing of associated resources. The algorithm uses a combined heuristic and meta heuristic technique to search solution space and optimize trade-off between cost, execution time. Empirical studies have shown that the algorithm significantly reduces total costs compared to other current baseline methods, while achieving acceptable execution times.

The author [14] shows a novel working scheduling approach in the fog cloud layers. The authors describe a fuzzy logic-based strategy for the inadequacy of conventional scheduling algorithms in handling uncertainties and dynamic situations. In this method, fuzzy rules and membership functions are proposed to evaluate and grade activities for minimization of task latency and energy consumption. This paper's system similarly distributes tasks within the

fog/cloud resources in a dynamic manner, depending on nature of the task urgency and demand for resources. The gargantuan simulation results achieved validate that the fuzzy-based scheduler gives optimized performance compared to traditional techniques, remaining very flexible and effective in most environments. The resultant study showed that the multi-objective fuzzy technique has been successful in dealing with the complexities in task scheduling in fog cloud computing.

The author [15] work propose a fog computing task scheduling that minimizes the total time taken to accomplish the tasks and energy, taking into consideration maximum resource use. This is an approach that uses algorithms like GA and PSO in assisting the fog nodes to perform their functions better since they are distributed in nature and have limitations with regard to resources. The use of models in energy and delay supports making scheduling decisions, coupled with continuous monitoring of resources. Results indicated that it is suitable for fog computing.

The author [16] introduces a new task scheduling method which takes the help of foraging behavior of African vultures to optimize the fog-cloud environment. The E-AVOA algorithm provides a proper improvement of certain important parameters like task completion time, energy use, and resource use. It works better than other algorithms. This paper gives an explanation on what to do and guides on how to make fog-cloud computing more efficient and sustainable.

The goal of DLJSF by the author [17] was to minimize latency and improve system efficiency through data locality optimization in job allocation under fog-cloud environments. It follows the hierarchy of the fog-cloud infrastructure to place jobs close to the sources so that their data transfer times and the utilization of network bandwidth are optimized. DLJSF integrates both heuristic and machine learning methods for dynamic assessment of the most efficient job allocations. The system considers parameters, such as the present state of the network, availability of processing resources, and specific needs towards data localization. Extensive simulation studies to evaluate the framework show substantial improvements in job completion time and resource use with respect to conventional scheduling methods.

The author [18] proposes an improved life-choice-based optimization approach (ILCO) for optimal scheduling of the work in fog-cloud blockchain (FCB) environments. The FCB network efficiently tackles security issues through blockchain technology, and the ILCO algorithm optimizes power, latency, and cost. The ILCO proposed has been shown to outperform state-of-art algorithms in convergence velocity, sustainability of population diversity. A multi-layered approach is used to model the system comprising sensors, fog, cloud, and blockchain. Fog broker is an important member of the scheduling family. Experimental results demonstrate that, compared with other methods, it scales up to optimize the performance metrics for IoT applications.

In [19] article “Meta Heuristic Algorithms in Task Scheduling for Cloud-Fog Computing: A Future Perspective” investigates the effectiveness of a number of MH algorithms on enhancing task scheduling in cloud-fogging environments. It outlines benefits, such as those of genetic techniques, particle swarm optimization, and ant colony optimization in resource allocations, energy efficiency, and real-time requirements. The report looks at major challenges, ranging from differences in resources, the ability to scale up, and the security of data.

In [20] author proposed a novel approach, namely two-stage task scheduling, to solve issues that currently take place in smart home environments by utilizing fog computing. In addition, Naive Bayes machine-learning model for training and a hybrid optimization technique comprising ACO and PSO deployed for task optimization. The proposed system will ultimately improve network efficiency, reduce time delay, and keep energy consumption at the lowest possible level. Performance comparison shows that the iFogSim simulator has significantly improved energy economy, latency reduction, and network utilization optimization compared to conventional approaches.

In [21] author introduces a novel task scheduling architecture to address issue in IoT environments. In turn, it introduces the DBC system represented by the IBC system and RBC system for ensuring security and efficiency. For optimization of task scheduling with regard to energy consumption and task deadlines, two algorithms, DBRP and DBAACS, are developed. Extensive experiments demonstrate the outperformance of these algorithms over the existing ones and, therefore, an improvement in task completion rates, makespan, energy consumption, adherence to deadlines. Collaborative networks between clouds and fogs using this DBC-enabled model effectively improve performance and trustworthiness in the presence of resource constraints, security issues in the fog nodes.

In [22] author introduces the Golden Eagle Whale Optimization (GEWO) technique for task scheduling in Cloud-Fog Computing (CFC). GEWO combines Golden Eagle Optimization (GEO) and Whale Optimization Algorithm (WOA) to scale the global exploration and local exploitation. Designed to handle the dynamic nature of CFC, GEWO optimizes task allocation across cloud, fog, and edge layers. Experiments show GEWO achieves a 26% reduction in task completion time, a 32% improvement in resource utilization, and a 29% increase in energy efficiency.

In [23] author introduces the Mexican Axolotl Optimization (MAO) algorithm for scheduling in Cloud-Fog Computing (CFC). It was formulated based on axolotl's regenerative abilities, MAO incorporates advanced heuristics and self-optimization mechanisms. It aims to improve resource allocation, reduce latency, and enhance efficiency by dynamically allocating tasks based on factors like makespan, success rate, and response time. Experimental results using the CloudSim framework show that MAO reduces makespan by 28%, response time by 36%, and increases success rate

by 31% compared to traditional algorithms. This makes MAO solution for optimizing task scheduling in dynamic CFC environments.

In [24] author has proposed an MGWO technique for job scheduling in cloud-fog environments. Proposed algorithm tries to achieve a balance scale two conflicting objectives minimizing transmission latency and minimizing energy consumption. The system works under a fog broker that maps work to fog nodes and cloud servers according to the characteristics of these entities. The MGWO algorithm contains steps that mimic the behavior of gray wolves, which are tracking, surrounding, hunting, and attacking the prey. Simulation results recorded using MATLAB showed that the Multi-objective Grey Wolf Optimizer (MGWO) performed better than other algorithms as it minimizes both delay and energy.

In [25] author highlights the challenges of 5G, B5G, and 6G technologies because, with advanced technological requirements and capabilities for mobile devices, the computational power of these devices is now checked by both the short lifecycle and inferior hardware. This paper contributes majorly toward addressing these inefficiencies associated with multi-tier computing networks (MTCNs), thus limiting it to cloud, fog, and edge layers, consequently enabling intelligent transportation systems (ITS) and other intelligent applications. This will be by developing a dynamic resource scheduling model based on the VNE and implemented through deep reinforcement learning to make resources more reconfigurable and efficient. The problem is tackled by modeling deep reinforcement learning-based scheduling for MTCNs that can optimize resource allocation through a mix of reward and gradient descent techniques. The proposed algorithm outperforms all existing baselines by 19.042%, 2.563%, 3.932% in condition of average revenue, revenue-cost ratio, VNR acceptance rate. Extensive simulations have confirmed that proposed algorithm is superior in utilizing resources, revenue generation, and VNR acceptance rate. This paper contributes to MTCNs multi-target-constrained networks and proposes a robust DRL-based VNE strategy.

In [26] author examines various task scheduling methodologies in fog computing aimed at enhancing resource utilization, energy efficiency, and task execution. The review encompasses priority-based scheduling, greedy, metaheuristic, learning-based and nature-inspired heuristic approaches, evaluating their respective advantages and drawbacks. Key challenges highlighted include dynamic environments, system heterogeneity, availability, resource limitations, and the transparency of algorithms.

In [27] author addresses the critical issue of managing the increasing data requests from IoT devices, which often lead to congestion and delays in cloud data centers. By leveraging fog computing, which processes data at network edge, proposed Reinforcement Learning Fog Scheduling (RLFS) algorithm aims to enhance load

balancing, response time, minimize energy consumption. RLFS algorithm employs a two-level reinforcement learning approach: the first level schedules real-time and important requests within local fog nodes, while the second level selects appropriate fog nodes when local resources are insufficient. Experimental results indicate that RLFS significantly improves load balance, response times compared to existing algorithms like LBSSA and DRAM. Moreover, RLFS optimizes resource utilization, resulting in fewer devices being used for task processing. Despite the increased execution time due to the complexity of reinforcement learning, the overall performance benefits outweigh this drawback. The study highlights the potential of RLFS in achieving efficient and reliable task scheduling in edge-fog-cloud environments, making it a promising solution for future of IoT networks.

In [28] author presents a hybrid optimization strategy that integrates a Genetic Algorithm (GA) with a modified Particle Swarm Optimization technique to enhance workflow scheduling in cloud-fog environments. The objective is to optimize latency, energy usage, cost, and quality of service (QoS). The hybrid method leverages GA for initial population refinement and modified PSO for quick convergence to optimal solutions. Evaluated using the WorkflowSim simulator on three scientific workflows (Cybershake, Montage, SIPHT), the proposed method outperforms traditional PSO, GA, GA-PSO, and Differential Evolution (DE) in reducing energy consumption, makespan, and execution costs. The study concludes that the hybrid approach effectively balances workloads and improves performance metrics in cloud-fog computing environments.

In [29] author introduces RAPTS, a task scheduling method aimed at improving efficiency in fog computing. RAPTS categorizes tasks into three priority queues based on deadlines and virtual machine (VM) processing delays, subsequently executing them using a Minimum Completion Time (MCT) algorithm. Evaluated through iFogSim, RAPTS demonstrates notable enhancements in resource utilization, response time, latency, cost, and task deadline adherence when compared to leading schedulers such as RACE (CFP) and RACE (FOP). Key findings indicate improvements of up to 53% in response time and 43% in task deadline adherence, showcasing the effectiveness of RAPTS in optimizing fog computing performance.

In [30] author proposes semi-dynamic real-time task scheduling algorithm for IoT-based applications. It addresses challenge of managing vast data generated by IoT devices, which traditional cloud data centers struggle to handle, especially for delay-sensitive applications. The algorithm uses modified genetic algorithm to optimize task scheduling by minimizing execution time and resource costs. It periodically assigns tasks to VMs based on the best permutation of tasks. Extensive experiments demonstrate the algorithm superiority over traditional methods, achieving good balance between makespan, execution cost while reducing task failure

rates. It effectively enhances IoT task management, offering significant improvements in performance and cost-efficiency.

In [31] author introduces a fog computing framework for smart factories, using Kubernetes for automatic deployment of containerized applications. Tasks are classified into Real-Time, General, and Storage categories, and fog nodes are categorized accordingly. It proposes the IDGSA to optimize task scheduling and resource usage. IDGSA-P extends this to cloud and fog collaborative computing by incorporating a penalty factor. Simulation results show IDGSA reduces data processing time by 50%, improves resource utilization by 60% compared to Kubernetes' default algorithm. IDGSA-P further reduces total task delay by 18%, 7% compared to only-cloud-fog computing. The study demonstrates significant improvements in task scheduling for smart factories.

In [32] author presents a decentralized heuristic algorithm for scheduling IoT tasks in fog-cloud systems. The goal is to meet Quality of Service (QoS) requirements, specifically tolerable latency, while improving application response times. The algorithm prioritizes scheduling tasks on the nearest fog nodes, balancing resource utilization and QoS. Evaluated using the iFogSim toolkit, the proposed approach outperforms existing methods like CASSIA-RR, LAMP, and Edgeward in reducing response time and optimizing resource.

In [33] author proposes the OSCAR model to enhance quality of service in fog-assisted IoT systems. It addresses bandwidth overhead, increased delay, and resource management issues by clustering tasks based on priority, deadline using the Expectation-Maximization algorithm. A modified heap-based optimizer schedules these tasks while the Categorical Deep Q Network (C-DQN) model manages resource allocation. The model also dynamically preempts tasks to prioritize those with shorter deadlines. The simulation work done by iFogsim tool

In [34] author introduces a star-quake operator to enhance Multi-Objective Gravitational Search Algorithm. This operator balances selection pressure, exploitation, and exploration to improve task scheduling by preventing local optima entrapment. Evaluated in fog-cloud scenarios, the algorithm optimizes makespan, energy consumption, and cost. Results show SQMOGSA outperforms existing methods, improving makespan by 18%, energy consumption by 22%, cost by 40%. It includes star-quake-enhanced MOGSA offers significant performance gains in IoT-based task scheduling. Table 1 summarizes all task scheduling techniques in fog computing which mentions limitations, addressed parameters. From Section II, it is evident that still there are open research areas to be addressed by authors for generation of task schedules precisely to fog nodes while addressing parameters fault tolerance, makespan, consumption of energy by using Fuzzy, reinforcement learning techniques.

III. SYSTEM MODEL

The Internet of Things (IoT) network is composed of three layers the edge layer, fog layer, and cloud layer. This layered

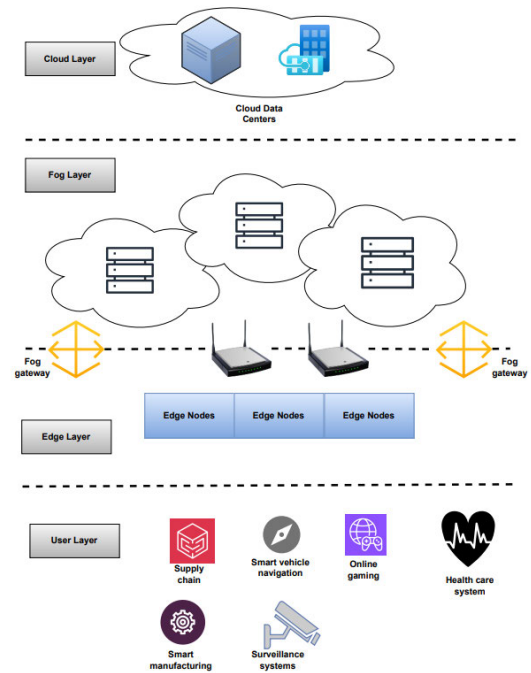


FIGURE 1. Architecture of fog computing.

architecture is specifically designed to effectively handle and process the large volumes of data produced by diverse IoT devices.

The **user layer** devices generates substantial amounts of multimedia data. These devices are categorized into various IoT subsections. Smart homes incorporate systems for surveillance and security, while smart healthcare includes devices for heart rate monitoring, glucose tracking, inhaler management, and real-time patient monitoring. Multimedia devices facilitate live video streaming and music playback, whereas smart logistics and supply chain systems focus on real-time routing, and navigation. Additionally implement traffic management systems while smart grids prioritize load forecasting for distribution circuits. Each data type generated by these devices has distinct characteristics, with scalar data typically being delay-tolerant and multimedia data being more sensitive to delays. This diversity requires significant storage capacity, high bandwidth, substantial processing power, and considerable energy consumption to manage the data effectively

The **edge Layer** is the part of IoT architectures and is situated between IoT devices and cloud or fog layers. It makes the processing localized, which means that computation can be done close to the data source, which will help in real-time decision-making as the latency rate is comparatively low. Reducing the time required for descriptive analytics directly on or near IoT devices guarantees quick responses that are vital in the implementation of IoT in applications such as self-driving automobiles and health monitoring. Besides, it refines information processes information in such a way

TABLE 1. Summary of task scheduling techniques in fog computing.

Ref	Technique	Limitations	Simulation Tool	Parameters
[11]	Reinforcement learning for task scheduling in fog computing	Limited to specific network states and environments	Python	Delay, energy consumption
[12]	Reinforcement learning algorithms, such as Genetic Algorithm and Q-learning for resource allocation	Focused on healthcare, might not generalize	MATLAB	Latency, make-span, throughput
[13]	Multi-objective task scheduling for Bot applications in a heterogeneous fog environment	Complexity in balancing multiple objectives	iFogsim	Cost, execution time
[14]	A fuzzy logic-based method for task scheduling approach in fog-cloud environments	Limited to scenarios modeled by fuzzy rules	workflowsim	Task delay, energy consumption
[15]	A multi-objective task scheduling technique that employs GA and PSO	Complexity in implementation	iFogsim	Task completion time, energy consumption, resource utilization
[16]	Enhanced African Vultures Optimization Algorithm (E-AVOA) for task scheduling in fog-cloud environments	Specific to fog-cloud, may not apply to other environments	iFogsim	Task completion time, energy consumption, resource utilization
[17]	Data-Locality Aware Job Scheduling Framework (DLJSF)	Focused on data locality, may not generalize	Cloudsim	Latency, system efficiency
[18]	Improved life-choice-based optimization algorithm (ILCO) for fog-cloud blockchain environments	Complexity in integrating blockchain	Python	Power, latency, cost
[19]	Metaheuristic algorithms for task scheduling in cloud-fog computing	Convergence speed, handling dynamic environments	Cloudsim	Resource allocation, energy efficiency, real-time requirements
[20]	Two-stage task scheduling approach using naive Bayes and hybrid optimization (ACO, PSO) for smart homes	Focused on smart homes, may not generalize	iFogSim	Energy efficiency, latency, network usage
[21]	Double blockchain system (IBC, RBC) for task scheduling in IoT environments	Complexity in blockchain integration	MATLAB 2020a	Energy consumption, task deadlines
[22]	Golden Eagle Whale Optimization (GEWO)	Focused on dynamic environments	Cloudsim	Task completion time, resource utilization, energy efficiency
[23]	Mexican Axolotl Optimization (MAO) algorithm for task scheduling in Cloud-Fog Computing	Limited to specific scenarios	CloudSim	Makespan, response time, success rate
[24]	Multi-objective Grey Wolf Optimizer (MGWO)	Limited objectives	MATLAB	Transmission delay, energy consumption
[25]	DRL-based dynamic resource scheduling for Multi-tier Computing Networks	Limited to specific network configurations	Pycharm	Revenue, revenue-cost ratio, VNR acceptance rate
[26]	Review of heuristic task scheduling methodologies in fog computing	General limitations of heuristic approaches	Reviewed MATLAB, Python, clousim, ifogsim	Resource utilization, energy efficiency, task execution
[27]	Reinforcement Learning Fog Scheduling (RLFS) algorithm for load balancing in edge-fog-cloud environments	Increased execution time due to RL complexity	Not specified	Load balance, response time, energy consumption
[28]	Hybrid optimization (GA, PSO) for workflow scheduling in cloud-fog environments	Complexity in hybrid approach	WorkflowSim	Makespan, energy consumption, cost
[29]	Resource Aware Prioritized Task Scheduling (RAPTS) for fog computing	Specific to heterogeneous fog environments	iFogSim	Resource utilization, response time, makespan, cost, task deadline adherence
[30]	Semi-dynamic real-time task scheduling for IoT-based applications in cloud-fog environments	Limited to specific real-time applications	MATLAB 2018b	Execution time, resource costs
[31]	IDGSA for task scheduling in smart factories using Kubernetes	Focused on smart factories	Python	Data processing time, resource utilization, total task delay
[32]	Decentralized heuristic algorithm for scheduling IoT tasks in fog-cloud systems	Limited to latency-sensitive applications	iFogSim	Response time, resource utilization
[33]	OSCAR model for QoS in fog-assisted IoT systems	Specific to dynamic IoT tasks	iFogSim	Response time, resource utilization, makespan time, throughput
[34]	Star-Quake enhanced Multi-Objective Gravitational Search Algorithm (SQ-MOGSA) for IoT-based task scheduling	Limited to fog-cloud scenarios	MATLAB 2018b	Makespan, energy consumption, cost

that only useful data gets pushed to the cloud, thus optimizing usage of the bandwidth thus also boosting data transfer rates. This way of functioning also localizes the application of networks and computations that make the best of cloud space, while at the same time provides a localized solution for security and privacy by reducing the transmission of sensitive data to the cloud. Additionally, the edge layer can utilize contextual data including geographical and meteorological—thus allowing it to provide appropriate and personalized services by improving user experiences in different applications. In conclusion, it can be asserted that the edge layer provide crucial contribution to the enhancement of IoT systems, and make possible the requirement of using modern solutions in the world of modern data.

The **fog layer**, positioned between the user and cloud layers, consists of various fog devices, including computing and storage. These devices can be either centrally located or geographically distributed. The strategic placement of fog nodes at the network edge allows users to connect with them via a single hop, contrasting with the multiple hops required to access cloud servers. This proximity significantly reduces data transmission latency and alleviates the burden on the network core. Fog computing not only enhances overall network performance but also offers location-specific information, services, and applications. This capability makes fog computing particularly well-suited for real-time processing needs within the IoT, as it can tailor responses to the specific context of the data origin, thereby providing more relevant and efficient service delivery.

Cloud layer comprises one or more cloud data centers provide extensive and on-demand access to virtually unlimited computational resources. These cloud data centers facilitate the offloading of intensive processing tasks from the fog layer, thereby enabling more efficient handling of complex computations and large-scale data storage. The cloud layer's robust infrastructure supports scalability and flexibility, allowing for the seamless execution of tasks that require significant processing power and storage capacity. This top-tier layer ensures that the overall IoT network operates efficiently, handling vast amounts of data with high reliability and performance. This hierarchical architecture, integrating the user, fog, and cloud layers, optimizes the management and processing of data within the IoT network. By leveraging the strengths of each layer, the network can deliver high performance, low latency, and context-aware services, making it a powerful framework for modern medical and healthcare applications.

IV. MATHEMATICAL MODELING AND SYSTEM ARCHITECTURE OF TASK SCHEDULING

In this mathematical model, we assume the presence of resources distributed across fog environments. Initially, to determine task priorities, it is crucial to assess the current workload on virtual machines (VMs). The workload on VMs

TABLE 2. Notations used in HTSFFDRL model.

Notation	Description
s_t	State at time t , representing environmental conditions and task-specific details
a_t	Action at time t , representing the selected fog node for task allocation
R_t	Reward at time t , integrating task completion, resource utilization, energy efficiency, cost, and fault tolerance
γ	Discount factor, adjusting the importance of future rewards
L_{actor}	Actor loss, computed through policy gradient methods to drive the actor network towards actions yielding higher rewards
L_{critic}	Critic loss, updating the value function to better estimate future rewards
T	Task length
P	Task priority
CPU_i	Current CPU utilization at fog node i
Mem_i	Available memory at fog node i
BW_i	Bandwidth at fog node i
E_i	Current energy level at fog node i
S	CPU speed
C_f	Cost factor
F_f	Fault factor
L_t	Task length in seconds, indicating the computational workload
D_t	Task deadline, dictating the maximum permissible completion time
R_t	Resource demands of the task, including CPU and memory requirements
θ_π	Parameters of the policy network
θ_v	Parameters of the value network
$h_{\theta_\pi}(s_t, a_t)$	Action-value function of the actor network

across these environments is calculated using (1):

$$wl_{VM,n1} = \sum_{w=1}^{n1} wl_{w,n1} \quad (1)$$

Once the current VM workload is determined, as VMs reside on physical machines (PMs), we compute the workload on each PM within the considered physical machines across fog environments using (2):

$$wl_{PM,i1} = \frac{wl_{VM,n1}}{PM_{i1}} \quad (2)$$

Understanding the processing capacities of VMs in different fog environments is essential since task priorities depend on these capacities. Processing capacity of a VM is calculated using (3):

$$prc_{VM,n1} = prno \times prnips \quad (3)$$

After determining ability of each VM, total processing capacity of $n1$ VMs considered across fog environments is computed using (4):

$$totprc_{VM,n1} = \sum_{VM=1}^{n1} prc_{VM,n1} \quad (4)$$

Our research requires the selection of certain virtual machines (VMs) in order to prioritize tasks. Efficiently

prioritizing activities is an essential aspect of task scheduling. Many classical scheduling systems allocate priority based on predetermined weights of fixed characteristics related to task features, such as bandwidth requirements, execution length, and memory use. Nevertheless, the use of fuzzy logic embodies a very adaptable and practical approach. We used Takagi-Sugeno fuzzy inference system, first developed by Takagi, Sugeno, and Kang, which provides a systematic mathematical foundation for constructing fuzzy rules.

The Takagi-Sugeno method itself from other fuzzy techniques like Mamdani and Tsukamoto, due to its numerous advantages, such as faster processing speed, greater computational efficiency, improved accuracy, and adaptability to various optimization methods. The reason for its larger size is that it replaces the time-consuming diversification procedure with a more efficient weighted average technique. You can represent the Sugeno model in following way:

Rule i: If X_1 is A_{i1} and ... and X_n is A_{in}

$$y_i = C_{i0} + C_{i1}X_1 + C_{i2}X_2 + \dots + C_{in}X_n \quad (5)$$

In fuzzy logic systems, for each rule i (where $i = 1, 2, 3, \dots, n$ and n is the total number of fuzzy rules), the input X_n is evaluated against a fuzzy set A_{in} . Based on this evaluation, the system computes an output y_i using the consequence parameters C_{in} .

The output of the Sugeno fuzzy model is calculated as

$$y = \frac{\sum_{i=1}^n w_i y_i}{\sum_{i=1}^n w_i} \quad (6)$$

where w_i is the weight of the i -th rule, which is determined by

$$w_i = \prod_{j=1}^n \mu_{ij}(X) \quad (7)$$

In this context, $\mu_{ij}(X)$ represents the membership function of X each input data value may correspond to multiple membership values. For instance, in the task prioritization process, an input parameter like memory consumption can be classified into low, medium, and high profiles with different membership degrees, such as 0.1, 0.5, and 0.9, respectively. These values lie within range of [0, 1]

In Table 3 High represented as H, Low represented as L, Medium represented as M. We developed the Takagi-Sugeno Fuzzy Logic Task Priority algorithm as a sophisticated technique to optimally order tasks in a sophisticated computer system. The system utilizes fuzzy logic to handle uncertainties and interdependencies among various attributes of work, such as task length (TL), memory need (MR), CPU use (CPU), and bandwidth (BW). Membership functions are utilized to describe each attribute of a task. Some input values are transformed into memberships. The membership functions map the input values to fuzzy categories like low, medium, and high that allow the algorithm to accommodate imprecision and unpredictability. A chain of if-then rules based on expert knowledge and empirical data gives foundation of the fuzzy rule for the algorithm. For example, a rule

TABLE 3. Fuzzy rules of sugeno inference system for task prioritization.

CPU	Task Length	Bandwidth	Memory	Priority
H	H	L	H	H
M	H	L	H	M
H	H	M	M	M
H	M	M	M	H
M	M	H	M	M
L	M	M	M	H
H	H	L	L	L
L	L	L	L	H
M	L	H	L	M
H	H	H	H	H
H	M	H	H	H
H	L	H	H	M
M	H	H	H	H
M	H	H	M	H

could state that if the value for MR is high, the value for CPU is high, the value for BW is low, and the value for TL is high, then the combination of those values determines the PR value. Each rule generates a linear equation to determine work priority based on input data.

$$PR = a_1 \times MR + b_1 \times CPU + c_1 \times BW + d_1 \times TL + e_1 \quad (8)$$

In the proposed fuzzy logic system, we utilize four input parameters: CPU utilization, Memory, Bandwidth, and Task Length, each of which can take one of three possible values: High (H), Medium (M), or Low (L). Based on these inputs, the system generates task priority as the output. Given that each input can have three possible values, the total number of fuzzy rules is calculated as $3^4 = 81$. This means our fuzzy logic system will generate 81 distinct rules to cover all possible combinations of the four input parameters and their associated output task priorities. Each rule is designed to evaluate a unique combination of the input parameters and map them to the corresponding task priority level, allowing the system to handle varying levels of resource availability and task requirements efficiently.

Task scheduling and using fuzzy logic the membership function remains equally critical in converting real data inputs into decision making qualitative values. These functions describe the level of the input parameter, to which it belongs to the predefined fuzzy sets, such as Low (L), Medium (M), High (H) as shown in Fig. 2. This consists of the cylinder number, the piston speed, and specific quantitative measurements of the engine that is mapped to a CPU utilization membership function. The following Fig. 2 represent membership function for the CPU utilization. This parameter indicates the levels of utilization of processor in the system at any given time. Low CPU membership function is a small output of 50%, Medium CPU ranges from 30% to 70%, and High CPU ranges from 50% to 100%. These triangular shaped functions show how the system discriminates various CPU usage levels based on the threshold values. Shown in Fig. 3 are the membership functions of memory utilization.

Algorithm 1 Takagi-Sugeno Fuzzy Logic Task Priority

Require: Data (or) Tasks D , CPU speed C_{speed} , Transfer time $T_{transfer}$

Ensure: Task priorities $Priorities$

```

1: Initialize file path  $D$ , CPU speed  $C_{speed}$ , Transfer time  $T_{transfer}$ 
2: Open file  $D$ , read lines, filter and convert to integer job sizes  $J$ 
3: Initialize empty list  $Jobs$ 
4: for each job  $j$  in  $J$  do
5:    $TL_j \leftarrow \frac{j}{C_{speed}}$ 
6:    $MR_j \leftarrow j \times 10^6 \times 4$ 
7:    $BW_j \leftarrow \frac{j \times 8}{T_{transfer}}$ 
8:   Append  $\{j, TL_j, MR_j, CPU_j, BW_j\}$  to  $Jobs$ 
9: end for
10: Define fuzzy variables  $TL, MR, CPU, BW, PR$ 
11: Define membership functions for  $TL, MR, CPU, BW, PR$ 

12: Establish fuzzy rules:
13: if  $MR$  is high and  $CPU$  is high and  $BW$  is low and  $TL$  is high then
14:    $PR = a_1 \times MR + b_1 \times CPU + c_1 \times BW + d_1 \times TL + e_1$ 
15: end if {Additional rules are omitted for brevity}
16: Initialize control system  $FIS$  with rules
17: Create simulation object  $Sim$  for  $FIS$ 
18: Initialize empty list  $Priorities$ 
19: for each job  $j$  in  $Jobs$  do
20:   Set inputs  $TL_j, MR_j, CPU_j, BW_j$ 
21:    $PR_j \leftarrow Sim.ComputePriority(TL_j, MR_j, CPU_j, BW_j)$ 
22:   Append  $PR_j$  to  $Priorities$ 
23: end for
24: Define function  $MapPriority(PR)$  to map numerical to categorical priorities
25: Apply  $MapPriority$  to  $Priorities$ 

```

This parameter evaluates the extent of RAM in use within the Cloud. Low Memory range to 32GB and shows usage as low while the Medium Memory range between 16GB to 48GB and High Memory is capped at 64GB. These functions demonstrate how the fuzzy logic system breaks the memory availability into different levels and, therefore, how it determines task importance and resource allocation.

Fig. 4 below shows the membership functions of bandwidth availability. This parameter determine the ability of the network to carry data at certain rate.. The Low Bandwidth function is valid up to 500 Mbps which shows low capacity of bandwidth, Medium Bandwidth up to 700 Mbps and High Bandwidth till up to 1000 Mbps. These functions exemplify how the system assesses network competencies essential for enhancing operations relying on massive amounts of data in fog computing paradigms. In Fig. 5 membership functions of the task length are presented. This parameter determines the time needed to do a particular task. Specifically, Low Task Length ranges from 0ms to 10,000ms which shows

that the tasks are short, Medium Task length covers from 5000ms to 15000ms while high cover 20000ms. These functions demonstrate how the fuzzy logic system describes the task workload, which may help balance job loads among teams. The membership functions it is easy to deal with uncertainty and variation in the cases of task scheduling. By mapping the entering input parameters into fuzzy sets, the system can make accurate decision for such performance factors as makespan, energy usage, and resource usage in a complicated computing climate.

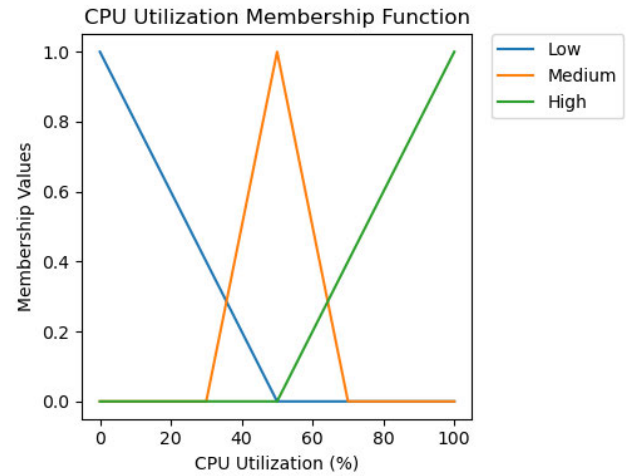


FIGURE 2. CPU utilization membership function.

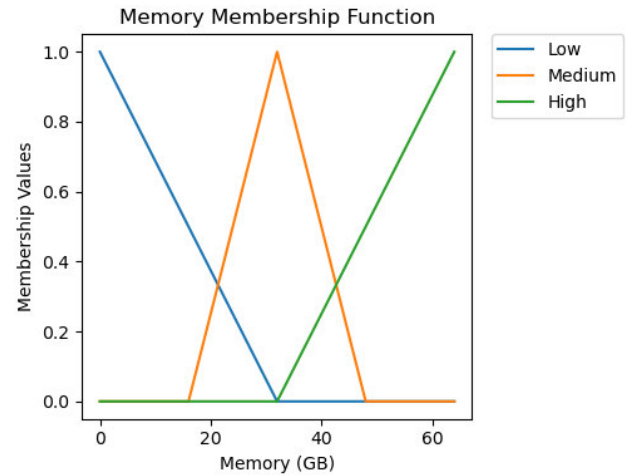


FIGURE 3. Memory membership function.

The defuzzification plays a crucial role by converting fuzzy outputs into crisp decisions, thereby enhancing resource investment and task scheduling. The defuzzification method employs the centroid approach, which calculates the centroid of a fuzzy output after aggregation. This method considers the entire range of fuzzy values rather than focusing solely on the highest membership level, improving the system's ability to identify a high-quality state. The defuzzification

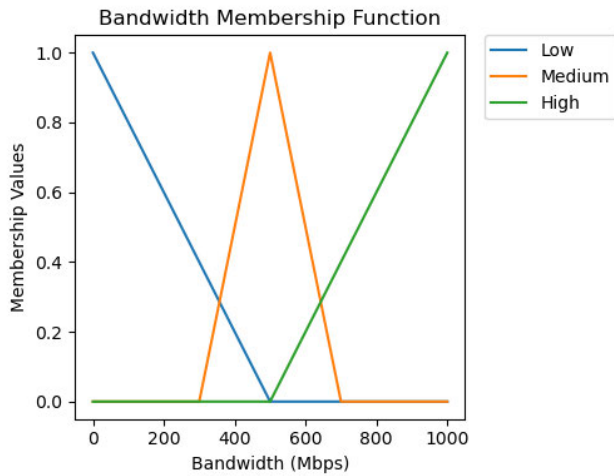


FIGURE 4. Bandwidth membership function.

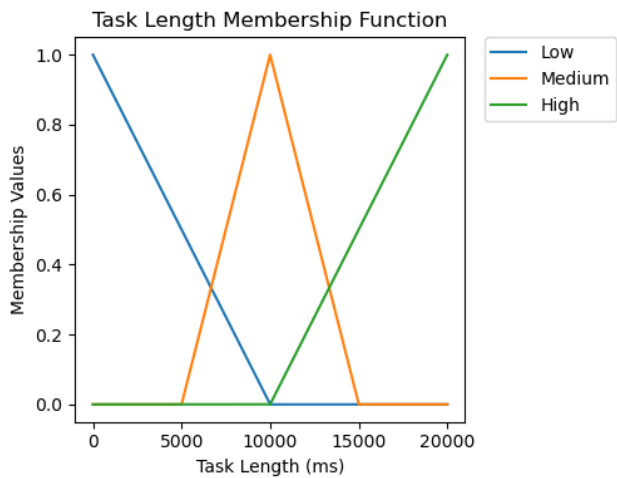


FIGURE 5. Task Length Membership Function.

process begins with the application of fuzzy inference rules to the input variables, producing fuzzy outputs across three categories: Negative, Neutral, and Positive. The Negative membership function indicates low CPU usage, low memory levels, and low bandwidth, signifying resource scarcity. For instance, if CPU usage exceeds 80% and available memory drops below 20%, tasks are assigned low priority, leading to delays. In contrast, the Neutral membership function describes moderate conditions, with CPU usage at 40-70% and memory at 20-50%. In this state, tasks can continue execution at a standard priority; for example, if CPU usage is at 60% and memory at 30%, normal scheduling.

The Positive membership function activates when CPU utilization is below 40%, memory load is above 50%, and bandwidth is high, indicating ample resources. For example, if CPU utilization is at 30% with sufficient memory, many tasks can be processed immediately at high priority. The superposition of these membership functions results

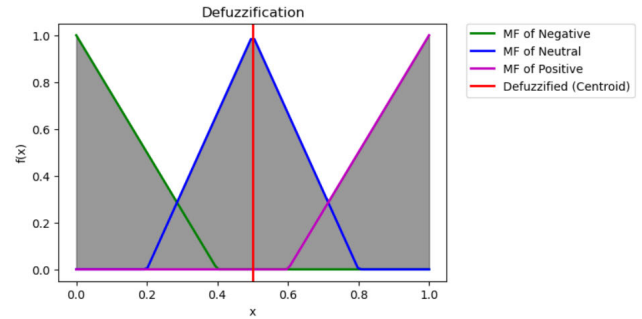


FIGURE 6. Defuzzification.

in an aggregated fuzzy output, which the centroid method defuzzifies by finding the mean of the fuzzy set. The visual representation of defuzzification shown in Fig. 6

Then convert these generated priority numbers into more understandable and manageable category priorities. In the planning process, we can easily set number importance to either “high,” “medium,” or “low” values under these classes and assist us in deciding on job scheduling. Most of the time, the Takagi-Sugeno Fuzzy Logic Task Priority method is very helpful in solving effective and reliable planning of work for cases like cloud computing, IoT networks, and real-time processing systems. Effective and reliable work planning involves controlling a lot of interconnected factors and adapting to changes in circumstances. This is implemented in the utilization of fuzzy logic for the complex task of organizing work, and it is achieved through enhanced system efficiency and resource utilization. This shows how fuzzy logic can be used practically as a useful tool in the most effective way of managing work.

Fuzzy logic is applied in the setting of the task priority and VM priority. These priorities are then used to set the time for new tasks to be performed by means of the HTSFFDRL task planner. Our combined planner includes HTSFFDRL and fuzzy, keeping the factors of makespan, cost, energy use, and fault tolerance as low as possible. Among important factors that influence the makespan are the working power of the VM as well as the memory needs of tasks. VMs with fast CPUs will execute orders rapidly, in such a way that if the tasks are to be loaded onto the memory for processing, the loading speed is fast and can be done without much delay. This combination leads to a shorter makespan because the tasks are executed more efficiently, minimizing the total time required for completion. Let $\mu_{\text{CPU, high}}$ represent the membership value of the CPU speed being high, and $\mu_{\text{Memory, low}}$ represent the membership value of the memory requirement being low. The degree to which the rule is satisfied (w_1) can be calculated as:

$$w_1 = \min(\mu_{\text{CPU, high}}, \mu_{\text{Memory, low}}) \quad (9)$$

The makespan can then be estimated by applying this rule weight to a consequent function representing the task

length (T) and priority (P):

$$\text{Makespan} = w_1 \times (T \times P) \quad (10)$$

Energy consumption is directly related to both the duration of task execution and the data transfer rates. High network bandwidth allows for fast data transfer, but if the task length is long, the VM and network components remain active for increase periods, leading to increased energy consumption. The continuous operation of these components over a prolonged duration results in higher energy usage. Let $\mu_{\text{Bandwidth, high}}$ represent the membership value of the bandwidth being high, and $\mu_{\text{Task, long}}$ represent the membership value of the task length being long. The degree to which the rule is satisfied (w_2) can be calculated as:

$$w_2 = \min(\mu_{\text{Bandwidth, high}}, \mu_{\text{Task, long}}) \quad (11)$$

The energy consumption can then be estimated by applying this rule weight to a function involving the task length (T), CPU speed (S), and priority (P):

$$\text{Energy Consumption} = w_2 \times (T \times S \times P) \quad (12)$$

The cost of task execution is influenced by both the processing speed and the duration of the task. When a VM has a low CPU speed, it processes tasks more slowly. If the task length is also long, the tasks take even longer to complete, increasing the resource usage and associated costs. Extended use of computing resources leads to higher costs due to prolonged occupation and potential usage-based billing. Let $\mu_{\text{CPU, low}}$ represent the membership value of the CPU speed being low, and $\mu_{\text{Task, long}}$ represent the membership value of the task length being long. The degree to which the rule is satisfied (w_3) can be calculated as:

$$w_3 = \min(\mu_{\text{CPU, low}}, \mu_{\text{Task, long}}) \quad (13)$$

The cost can then be estimated by applying this rule weight to a function involving the task length (T), CPU speed (S), a cost factor (C_f), and priority (P):

$$\text{Cost} = w_3 \times (T \times S \times C_f \times P) \quad (14)$$

The probability of faults occurring during task execution increases when the required memory is high and the available network bandwidth is low. High memory requirements can lead to resource contention, especially if multiple tasks compete for limited memory. Low bandwidth can exacerbate the situation by creating bottlenecks in data transfer, increasing the likelihood of errors, timeouts, and other faults due to delayed or incomplete data processing. Let $\mu_{\text{Memory, high}}$ represent the membership value of the memory requirement being high, and $\mu_{\text{Bandwidth, low}}$ represent the membership value of the bandwidth being low. The degree to which the rule is satisfied (w_4) can be calculated as:

$$w_4 = \min(\mu_{\text{Memory, high}}, \mu_{\text{Bandwidth, low}}) \quad (15)$$

The fault probability can then be estimated by applying this rule weight to a function involving the task length (T), CPU speed (S), a fault factor (F_f), and priority (P)

$$\text{Fault Probability} = w_4 \times (S \div T \times F_f \times P) \quad (16)$$

These fuzzy rules, by integrating input variables through fuzzy logic, enable a more nuanced and adaptive approach to task scheduling, addressing the complexity and variability of real-world computing environments.

V. PROPOSED METHODOLOGY

This section proposed HTSFFDRL algorithm methodology. Such a type of reinforcement learning-based approach optimizes fog computing's hybrid task scheduling technique through a combination with fuzzy logic. The HTSFFDRL algorithm is a novel hybrid task-scheduling technique in fog computing for effective delay minimization by employing fuzzy logic and reinforcement learning. The HTSFFDRL methodology includes strong asynchronous training techniques and actor-critic techniques shown in Fig7. It is particularly useful in scenarios with complex state and action spaces. With this reinforcement learning method, agents can directly interact with the environment without explicit direction to gain optimal scheduling strategies autonomously. The HTSFFDRL task scheduling method to allocate computing workloads to appropriate resources such as fog nodes or VMs in order to reduce energy consumption and maximize the cost-effectiveness of completion time. The dynamic and complex nature of these systems makes traditional scheduling approaches not feasible. Our study focuses on the transmission of data over a fully connected network and 256 neurons in each hidden layer. We set the network's stride to 1, and the kernel size to 2. The SoftMax activation function assures that the resulting values lie within the range of 0 to 1, after the data passes through these concealed layers. In fog computing, the hybrid task scheduling strategy makes use of a framework that combines reinforcement learning and fuzzy logic. This approach executes numerous agents concurrently on separate threads, creating a multi-threaded network where each agent operates independently. A worldwide network determines the incentives after each agent autonomously evaluates the results and transmits its conclusions. Parallel execution expedites the training phase of the approach by utilizing the allocated state space for each actor network. The HTSFFDRL algorithm employs a neural network based approach to determine the policy (actor) and the value function (critic). These are of paramount importance in reinforcement learning tasks because they facilitate action evaluation and decision formulation.

A. STATE SPACE S_T

In task scheduling within fog computing, the state s_t plays an important role by encapsulating the environmental conditions and task-specific details to control the flow of decision-making processes. This comprehensive state representation,

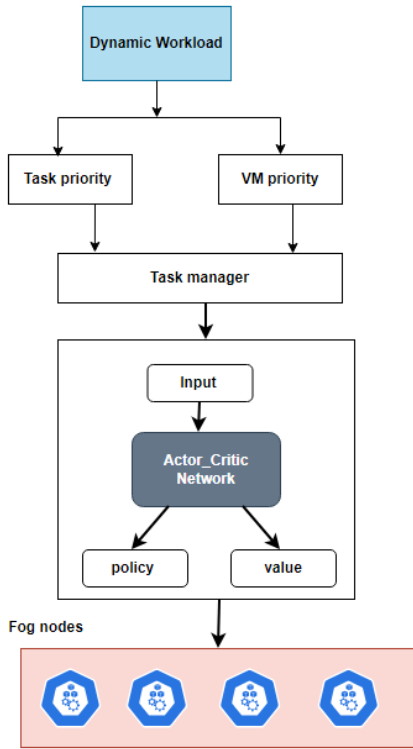


FIGURE 7. System model architecture.

denoted as:

$$s_t = [L_t, P_t, D_t, R_t, CPU_i, Mem_i, BW_i, E_i] \quad (17)$$

integrates the parameters essential for efficient allocation and scheduling strategies.

Firstly, L_t signifies the task length in seconds, indicating the computational workload required for execution. Secondly, P_t denotes the priority of the assigned task, influencing its scheduling priority based on criticality or predefined rules. Thirdly, D_t establishes the task deadline, marking the maximum permissible completion time to ensure timely execution.

Moreover, R_t specifies the resource demands of the task, including CPU and memory requirements crucial for effective allocation among the fog nodes. The parameters CPU_i , Mem_i , and BW_i reflect the current CPU usage, available memory, and bandwidth at fog node i individually, influencing the capacity to handle additional tasks without encountering resource constraints.

Lastly, E_i indicates the current level of energy consumption at fog node i , essential for implementing energy-aware scheduling strategies aimed at optimizing operational efficiency and extending node lifespan. Together, these parameters within s_t facilitate adaptive decision-making in dynamic and heterogeneous fog computing environments, ensuring tasks are allocated optimally to meet performance objectives and effectively manage resources.

B. ACTION (A_T)

The action a_t represents the decision made by the agent at time t , where $a_t \in \{1, 2, 3, 4, 5, 6\}$. This discrete action corresponds to selecting one of the available fog nodes to allocate a task. The choice of a_t is a critical aspect of the task scheduling process in fog computing, seeking to enhance a variety of performance metrics, including energy efficiency, latency, and resource utilization. The selection of a_t is influenced by the current state s_t , which includes parameters like task length L_t , task priority P_t , and the resource availability and condition of each fog node i (CPU utilization CPU_i , available memory Mem_i , bandwidth BW_i , and energy level E_i). The goal is to maximize the system efficiency and meet task requirements effectively.

C. REWARD R_T

The reward function R_t integrates to guide optimal decision-making by the agent. The task completion reward $R_t^{completion}$ time of task execution, offering a positive reward for on-time completions and imposing a penalty for missed deadlines. The resource utilization reward $R_t^{utilization}$ promotes the efficient resource allocation across fog nodes, encouraging actions that minimizing the time and maximizing the throughput. Additionally, the energy efficiency reward R_t^{energy} motivating the agent to prioritize tasks that reduce energy usage or enhance the energy efficiency. The cost component R_t^{cost} consider the expense associated with task execution, aim to minimizing the operational expense while maintaining performance. It can be defined based on factors such as computational resources used, data transfer costs, and maintenance. Fault tolerance, R_t^{fault} , addresses the resilience of task scheduling by actions that lead to system failure or degraded performance. This component the agent to prioritize fault-tolerant strategies, such as load balancing across nodes, redundancy in task assignment, or fault detection in fog nodes.

The overall reward function is formulated as:

$$R_t = R_t^{completion} + R_t^{utilization} + R_t^{energy} + R_t^{cost} + R_t^{fault} \quad (18)$$

The agent optimized the task scheduling decisions in fog computing, considering time completion of resource efficiency, energy conservation, cost-effectiveness, and fault tolerance to improve overall system performance and reliability.

D. POLICY (π)

In task scheduling within fog computing, the policy $\pi(a_t | s_t; \theta_\pi)$ defines how actions a_t are probable selected based on the current state s_t :

$$\pi(a_t | s_t; \theta_\pi) = \frac{e^{h_{\theta_\pi}(s_t, a_t)}}{\sum_{b=1}^N e^{h_{\theta_\pi}(s_t, b)}} \quad (19)$$

Here, $h_{\theta_\pi}(s_t, a_t)$ represents the action-value function of the actor network, which computes the preference or score

Algorithm 2 HTSFFDRL for Task Scheduling

Require: Env(data, vms, action_space), Actor Critic(action_space), Actor optimizer, Adam optimizer with learning rate, Discount factor (gamma), Maximum steps per episode (max_steps_per_episode), Number of episodes (num_episodes)

Ensure: all_rewards, all_actor_losses, all_critic_losses, all_metrics, all_job_assignments

- 1: Initialize: all_rewards, all_actor_losses, all_critic_losses, all_metrics, all_job_assignments = []
- 2: **for** episode in range(num_episodes) **do**
- 3: state = env.reset() {Reset environment for new episode}
- 4: episode_reward = 0
- 5: actor_losses = []
- 6: critic_losses = []
- 7: **while** True **do**
- 8: action_probs, critic_value = ActorModel(state)
- 9: action = sample action from action_probs
- 10: next_state, reward, done, _ = env.step(action)
- 11: episode_reward += reward
- 12: _, next_critic_value = CriticModel(next_state)
- 13: target = reward + gamma * next_critic_value
- 14: advantage = target - critic_value
- 15: actor_loss = -log(action_probs[action]) * advantage
- 16: critic_loss = HuberLoss(critic_value, target)
- 17: **Compute gradients:**
- 18: actor_grads = ComputeGradient(actor_loss, Actor-Model.trainable_variables)
- 19: critic_grads = ComputeGradient(critic_loss, Critic-Model.trainable_variables)
- 20: **Apply gradients:**
- 21: ApplyGradients(actor_optimizer, actor_grads)
- 22: ApplyGradients(critic_optimizer, critic_grads)
- 23: **Append losses:**
- 24: actor_losses.append(actor_loss)
- 25: critic_losses.append(critic_loss)
- 26: state = next_state
- 27: **if** done or timestep ≥ max_steps_per_episode **then**
- 28: break
- 29: **end if**
- 30: **end while**
- 31: metrics = env.get_metrics()
- 32: **end for**

associated with taking action a_t in state s_t . This probabilistic selection mechanism ensures that actions are chosen with probabilities proportional to their evaluated scores, enabling adaptive decision-making that balances exploration and exploitation. This policy is crucial for guiding the agent (scheduler) in selecting fog nodes effectively, aiming to maximize rewards by leveraging past experiences and adapting to current environmental conditions dynamically.

E. VALUE FUNCTION

In task scheduling within reinforcement learning, the value function $V(s_t; \theta_v)$ serves a pivotal role in guiding decision-making processes. Specifically, s_t represents the current state of the system, encompassing factors such as task attributes (e.g., length, priority), resource availability (CPU, memory), and network conditions (bandwidth, energy levels). The value function estimates the expected return starting from state s_t , considering future rewards and penalties associated with task scheduling decisions.

$$V(s_t; \theta_v) = \mathbb{E}[R_{t+1} + \gamma V(s_{t+1}; \theta_v) \mid s_t, a_t] \quad (20)$$

Here R_{t+1} denotes the immediate reward obtained after executing action a_t in state s_t . For task scheduling, this could be a reward reflecting successful task completion within deadlines or penalties for missed deadlines. γ is the discount factor, adjusting the importance of future rewards relative to immediate rewards. $V(s_{t+1}; \theta_v)$ represents the expected return from the subsequent state s_{t+1} , which depends on the action a_t chosen in s_t . The value function guides the agent (such as scheduler) in making decisions that optimize task allocation and scheduling. By estimating future rewards, it helps prioritize actions that lead to efficient resource utilization, meet task deadlines, and minimize the metrics.

F. ACTOR-CRITIC TRAINING NETWORK

In task scheduling using reinforcement learning, the Actor-Critic training network stands as a sophisticated approach to iterative improve decision-making processes. This dual-network system refines scheduling policies by concurrently optimizing an actor network, responsible for action selection, and a critic network, which evaluates actions based on their outcomes.

The Actor Loss, computed through policy gradient methods, drives the actor network towards actions that maximize expected rewards:

$$L_{actor} = -\mathbb{E}_{a_t \sim \pi(\cdot \mid s_t; \theta_\pi)} [A(s_t, a_t) \log \pi(a_t \mid s_t; \theta_\pi)] \quad (21)$$

where $A(s_t, a_t)$ represents the advantage function, indicating how advantageous an action a_t is relative to the expected return.

Concurrently, the Critic Loss updates the value function $V(s_t; \theta_v)$ to better estimate future rewards, aiding in more informed decision-making:

$$L_{critic} = \frac{1}{2} \mathbb{E}_{s_t} [(V(s_t; \theta_v) - V_{target})^2] \quad (22)$$

where $V_{target} = R_{t+1} + \gamma V(s_{t+1}; \theta_v)$ serves as the Temporal Difference (TD) target, integrating the immediate reward R_{t+1} and the discounted future value $V(s_{t+1}; \theta_v)$.

This iterative learning process aligns the actor's policy with the critic's evaluation, fostering policies that balance exploration and exploitation for efficient task allocation and scheduling. The actor network learns to select actions probabilistically using the softmax function, ensuring adaptive

decision-making:

$$\pi(a_t|s_t; \theta_\pi) = \frac{e^{h_{\theta_\pi}(s_t, a_t)}}{\sum_{b=1}^N e^{h_{\theta_\pi}(s_t, b)}} \quad (23)$$

where $h_{\theta_\pi}(s_t, a_t)$ represents the action-value function output of the actor network for action a_t . By constant improvement of network, the system adjusted the dynamic changes in job parameters, resource availability, and network. This improves scheduling efficiency and achieves performance goals in challenging contexts like fog computing and IoT networks. The adaptive capacity improves both work completion rates and resource usage, while also promoting sustainable operation through energy consumption and cost minimization.

G. PARAMETERS UPDATING

This process was repeated gathering data and mapping tasks to appropriate VMs using the improved HTSFFDRL approach, aiming to maximize the reward. After collecting all information and adjusting actor and critic network parameters based on observed rewards and state transitions. This process uses techniques like stochastic gradient descent to optimize the actor's policy towards actions tolerant higher rewards and refine the critic value estimated for better future predictions. It balances exploration and exploitation to maximize cumulative rewards. Hyperparameters like learning rates and regularization ensure stable learning and prevent overfitting. In task scheduling environments, such updates optimize job allocations across resources, adapting to dynamic conditions and enhanced the overall system efficiency and performance.

The HTSFFDRL is designed to enhance task scheduling in fog computing by addressing the limitations of traditional models. It comprises three key components that work together to optimize performance. First HTSFFDRL employs a Takagi-Sugeno fuzzy inference system which dynamically assesses critical parameters such as CPU usage, memory requirements, and task length. This integration of fuzzy logic allows for real-time task prioritization, enabling the framework to adapt to the fluctuating demands of IoT workloads while improving scheduling accuracy in the face of uncertainties. Second, the framework utilizes an actor-critic reinforcement learning model that continuously learns from performance metrics, allowing HTSFFDRL to adjust its scheduling policies in real time. This ongoing adaptation ensures efficient resource utilization as workload conditions change. Lastly, HTSFFDRL incorporates a multi-objective reward function that integrates essential performance metrics, including makespan, energy efficiency, cost, and fault tolerance. This comprehensive approach enables the framework to make informed decisions that balance competing objectives, optimizing overall system performance while contributing to long-term operational sustainability.

VI. EXPERIMENTAL RESULTS

In this section, we discuss the simulation and results of the proposed Hybrid Task Scheduling Framework with Fuzzy

Deep Reinforcement Learning (HTSFFDRL). The complete simulation proposed approach is conducted using SimPy. This approach utilizes various datasets from the Google Cloud Jobs dataset, representing small, medium, and large datasets. section VI-A simulation setting used in HTSFFDRL model. section VI-B, VI-C, VI-C and VI-E represents the evaluation metrics makespan, energy, cost and fault tolerance. section VI-G represents the how many tasks are assigned in each episode of fog nodes. section VI-H analysis of simulation results compared against the baseline algorithms LSTM, DQN and A2C.

A. SIMULATION SETTINGS USING THE HTSFFDRL

In the HTSFFDRL algorithm, total time complexity encompasses several components. The priority levels of tasks, with 'low', 'medium', and 'high' priorities mapped to 0, 1, and 2, respectively. The data was then normalized using MinMaxScaler, which scaled the features to a range between 0 and 1, excluding the 'Job ID'. Subsequently, the normalized data was sorted based on priority in descending order. The time complexity for loading and preprocessing the dataset is $O(n \log n)$, where n is the number of tasks, due to the sorting operation. The core of the study involved the development of a TaskScheduling using the OpenAI Gym framework. This environment was designed to simulate the task scheduling process, where the state space consisted of the normalized task features, and the action space represented the available virtual machines (VMs). The environment tracked several performance metrics, including energy consumption, cost, faults, and makespan, which is the total time taken to complete all tasks. Each step in the environment involved assigning a task to a selected VM and updating the relevant metrics based on the task's characteristics and the VM's specifications. The time complexity of each step in the environment is $O(1)$ for action selection and $O(1)$ for metric updates, resulting in a total complexity of $O(n)$ for n tasks. Six different VMs were defined, each with varying CPU capacities, memory, and bandwidth.

The action space allowed for discrete selection of these VMs, while the observation space provided the normalized task features as input. The Actor-Critic model, implemented using TensorFlow, consisted of a shared dense layer followed by separate layers for the actor and critic networks. The actor network outputted probabilities for selecting each VM, while the critic network estimated the value of the current state. The complexity of the forward pass through the deep reinforcement learning is $O(m)$, where m is the number of neurons in the largest layer. The training process of the HTSFFDRL algorithm involved multiple episodes, where the model interacted with the environment to optimize task scheduling. The loss function was used for the critic, while the actor loss was computed based on the advantage function. Gradients for both actor and critic losses were computed and applied to update the model parameters. Throughout the training, metrics such as total reward, actor loss, and critic loss were recorded, alongside the specific assignments of

TABLE 4. Simulation settings.

Entities	Quantity
Total Number of Tasks	100–1000
Task Lengths	15000 – 900000
Storage of Host	3TB
Host RAM	32GB
Bandwidth	1000MBPS
Number of VMs	60
Bandwidth of VMs	200MBPS
Processor	Intel (i7)
Operating System	Linux

TABLE 5. Parameter settings.

Parameter	Value
Learning Rate (Actor)	0.001
Learning Rate (Critic)	0.001
Gamma	0.99
Max Steps per Episode	1000
Number of Episodes	100
Optimizer	Adam
Activation Function (Hidden Layers)	ReLU

tasks to VMs. The complexity of training per episode is $O(n \cdot m)$ due to the forward and backward passes for each task. The results indicated a clear improvement in task scheduling efficiency. Average metrics over 100 episodes showed significant optimization in makespan, energy consumption, cost, and faults. The training process consistent reduction in both actor and critic losses, reflecting the model learning rate. The visualizations of the total rewards, actor losses, and critic losses across episodes provided insights into the training dynamics. The distribution of tasks assigned to each fog node per episode was also plotted, the load balancing achieved by the model.

B. EVALUATION OF MAKESPAN BY HTSFFDRL

Evaluating makespan in fog computing task scheduling for optimizing resource use and minimizing task completion time, thereby enhancing system efficiency. It ensures balanced workload distribution across fog nodes reducing and improving the performance. We evaluating the makespan using the configuration settings in Table 4 assigning workloads from real-time datasets of Google Cloud Jobs [35] to our proposed model, HTSFFDRL. The datasets are categorized as small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks), with task lengths ranging from 15,000 to 900,000.

Table 6 presents the makespan results for the small dataset. HTSFFDRL was evaluated against baseline algorithms LSTM, DQN, and A2C over 50 iterations. The makespan values for LSTM (for 100 to 350 tasks) are 1471.80 to 3931.27 for DQN, they range from 1279.83 to 3418.49 for A2C, from 1141.99 to 3055.80 and for HTSFFDRL, they are 1057.4 to 2829.3. The result visualized in Fig. 8.

Table 7 displays the makespan for the medium dataset. The makespan results for LSTM (400-650 tasks) are 4665.00 to 7561.24 for DQN, 4047.82 to 6584.56; for A2C, 3614.41 to 5874.64 and for HTSFFDRL, 3345.2 to 5439.7. These results are shown in Fig. 9. Table 8 shows the makespan for large datasets. The makespan for LSTM (700-1000 tasks) is 8185.38 to 11095.82; for DQN, it ranges from 7122.07 to 9649.42 for A2C, from 6358.99 to 8623.23 and for HTSFFDRL, 5882.4 to 7986.4. The visual representation of these result shown in Fig. 10. The experimental results shown that HTSFFDRL consistently outperforms the LSTM, DQN, and A2C algorithms in terms of makespan.

TABLE 6. Makespan for small dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset100	1471.80	1279.83	1141.99	1057.4
Dataset150	1880.94	1635.59	1460.70	1352.5
Dataset200	2434.74	2122.38	1892.84	1751.7
Dataset250	2892.89	2515.12	2246.87	2079.9
Dataset300	3420.82	2974.63	2655.91	2456.4
Dataset350	3931.27	3418.49	3055.80	2829.3

TABLE 7. Makespan for medium dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset400	4665.00	4047.82	3614.41	3345.2
Dataset450	5116.61	4440.53	3964.22	3672.8
Dataset500	5852.53	5080.46	4538.49	4193.6
Dataset550	6353.67	5533.63	4931.82	4570.2
Dataset600	7094.62	6177.93	5516.55	5098.1
Dataset650	7561.24	6584.56	5874.64	5439.7

TABLE 8. Makespan for large dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset700	8185.38	7122.07	6358.99	5882.4
Dataset750	8515.62	7405.75	6614.06	6124.5
Dataset800	9007.01	7824.35	6981.02	6452.8
Dataset850	9570.64	8330.12	7437.79	6882.4
Dataset900	10121.53	8801.33	7859.23	7268.9
Dataset950	10529.46	9156.05	8175.86	7563.2
Dataset1000	11095.82	9649.42	8623.23	7986.4

C. EVALUATION OF ENERGY CONSUMPTION BY HTSFFDRL

Evaluating energy consumption in task scheduling is crucial for optimizing power use and enhancing energy efficiency, which leads to reduced operational costs. This evaluation helps balance the load across fog nodes, preventing overloading and ensuring sustainable resource utilization. Additionally, efficient energy consumption prolongs the lifespan of fog devices, enhancing reliability and availability, contributing to a more eco-friendly and cost-effective environment. We evaluating the energy consumption using

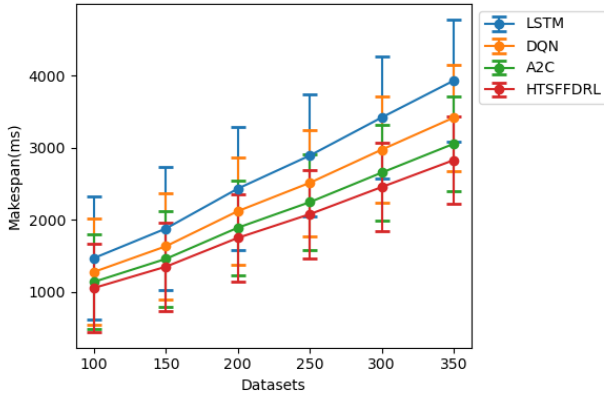


FIGURE 8. Makespan of small datasets.

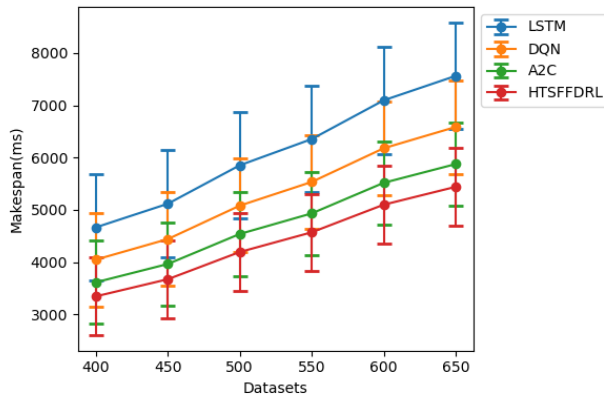


FIGURE 9. Makespan of medium datasets.

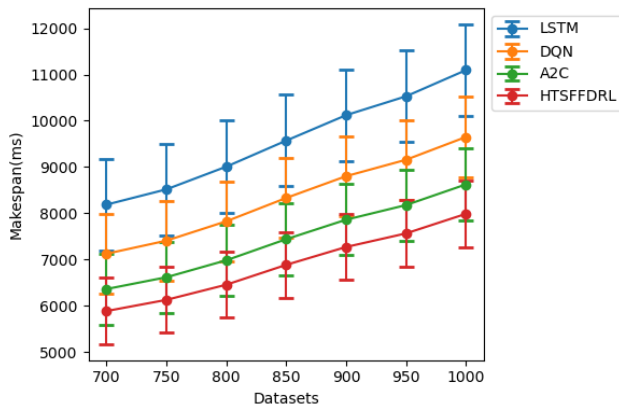


FIGURE 10. Makespan of large datasets.

the configuration settings in Table 4 assigning workloads from real-time datasets of Google Cloud Jobs [35] to our proposed model, HTSFFDRL. The datasets are categorized into small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks), with task lengths ranging from 15,000 to 900,000.

Table 9 presents energy consumption results for small datasets. After 50 iterations, HTSFFDRL was evaluated

against baseline algorithms LSTM, DQN, and A2C. The energy consumption for LSTM (100-350 tasks) is 28.17 to 99.18; for DQN, it ranges from 24.49 to 86.24; for A2C, from 20.88 to 77.44 and for HTSFFDRL, from 16.30 to 71.7. The results are visualized in Fig. 11. Table 10 shows energy consumption for medium datasets. The energy consumption for LSTM (400-650 tasks) is 113.93 to 192.37; for DQN, 98.22 to 167.26; for A2C, 87.68 to 149.04; and for HTSFFDRL, from 80.8 to 137.9. The results are shown in Fig. 12. Large Dataset: Table 11 depicts energy consumption for large datasets. The energy consumption for LSTM (700-1000 tasks) is 214.21 to 306.40; for DQN, 186.26 to 266.43; for A2C, 166.64 to 237.82; and for HTSFFDRL, from 154.3 to 219.8. The results are illustrated in Fig. 13. Our experimental results shown that HTSFFDRL significantly improves energy consumption compared to LSTM, DQN, and A2C algorithms.

TABLE 9. Consumption of energy (J) for small dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset100	28.17	24.49	20.88	16.30
Dataset150	46.74	40.98	36.07	31.40
Dataset200	53.81	46.78	40.78	36.80
Dataset250	71.36	62.58	48.53	41.60
Dataset300	85.34	74.22	66.20	60.30
Dataset350	99.18	86.24	77.44	71.70

TABLE 10. Consumption of energy (J) for medium dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset400	113.93	98.22	87.68	80.80
Dataset450	136.15	118.39	105.70	97.30
Dataset500	156.82	136.36	122.28	113.60
Dataset550	166.12	144.45	128.63	119.10
Dataset600	175.37	152.49	136.42	126.40
Dataset650	192.37	167.26	149.04	137.90

TABLE 11. Consumption of energy (J) for large dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset700	214.21	186.26	166.64	154.30
Dataset750	234.37	203.80	181.43	167.60
Dataset800	248.28	215.89	192.22	177.80
Dataset850	263.48	229.11	204.56	189.20
Dataset900	267.24	232.38	207.48	192.50
Dataset950	279.71	243.22	217.52	201.40
Dataset1000	306.40	266.43	237.82	219.80

D. EVALUATION OF COST BY HTSFFDRL

Evaluating task scheduling costs in fog computing is essential for optimizing resource allocation and reducing operational expenses. Efficient distribution of tasks across fog nodes minimizes costs and maximizes cost-effectiveness, leading to better financial management and a more sustainable computing environment. We calculate costs using configuration settings from Table 4, assigning workloads from Google Cloud Jobs [35] real-time datasets to our proposed

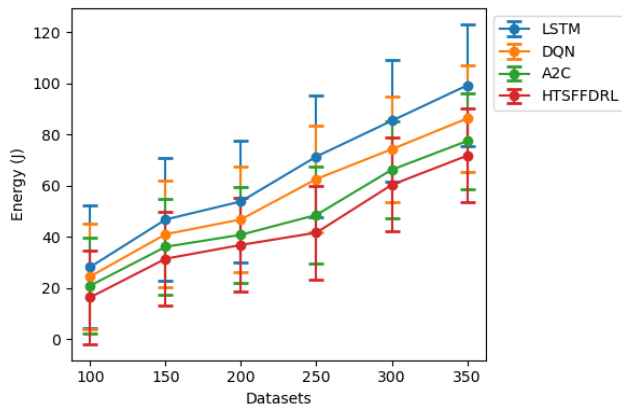


FIGURE 11. Consumption of energy for small datasets.

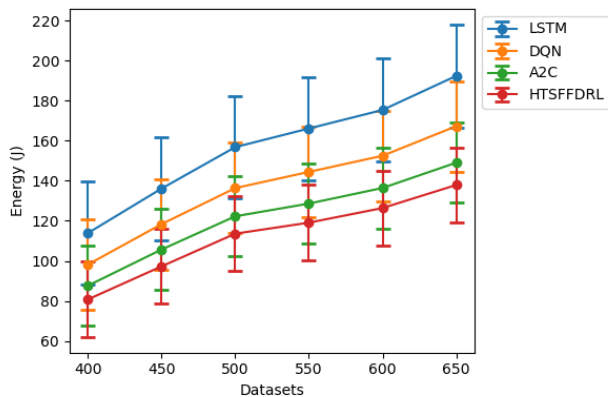


FIGURE 12. Consumption of energy for medium datasets.

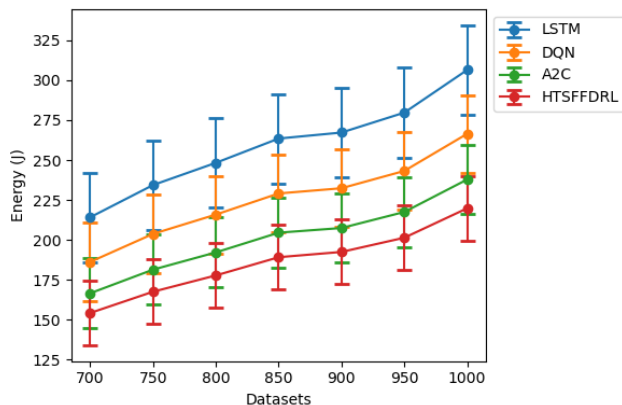


FIGURE 13. Consumption of energy for large datasets.

model, HTSFFDRL. The datasets are categorized into small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks), with task lengths ranging from 15,000 to 900,000.

Table 12 shows costs for small datasets. After 50 iterations, we evaluated HTSFFDRL against LSTM, DQN, and A2C algorithms. The costs for LSTM (100-350 tasks) range from 284.03 to 691.98; for DQN, from 247.84 to 601.72;

for A2C, from 220.00 to 537.78; and for HTSFFDRL, from 203.7 to 498.3. The results are shown in Fig. 14. Table 13 presents costs for medium datasets. The costs for LSTM (400-650 tasks) range from 788.78 to 1235.24; for DQN, from 685.89 to 1074.12; for A2C, from 612.58 to 955.46; and for HTSFFDRL, from 567.2 to 884.5. The result are shown in Fig. 15. Table 14 depicts costs for large datasets. The costs for LSTM (700-1000 tasks) range from 1305.24 to 1870.78 for DQN, from 1134.99 to 1626.77; for A2C, from 1014.27 to 1451.58; and for HTSFFDRL, from 938.4 to 1343.5. The result are shown Fig. 16. The results indicate that our proposed HTSFFDRL model improves cost efficiency compared to LSTM, DQN, and A2C algorithms.

TABLE 12. Cost for small dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset100	284.03	247.84	220.00	203.7
Dataset150	466.56	405.70	361.69	334.5
Dataset200	529.69	460.60	411.26	380.8
Dataset250	567.35	493.34	440.13	407.5
Dataset300	621.56	540.49	482.58	447.2
Dataset350	691.98	601.72	537.78	498.3

TABLE 13. Cost for medium dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset400	788.78	685.89	612.58	567.2
Dataset450	854.45	742.13	662.43	613.9
Dataset500	997.71	866.71	774.43	717.6
Dataset550	1113.69	968.42	865.40	801.3
Dataset600	1173.77	1020.67	911.58	844.6
Dataset650	1235.24	1074.12	955.46	884.5

TABLE 14. Cost for large dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset700	1305.24	1134.99	1014.27	938.4
Dataset750	1347.21	1171.49	1046.51	970.1
Dataset800	1589.44	1373.42	1224.02	1133.9
Dataset850	1656.62	1440.54	1286.20	1191.3
Dataset900	1696.77	1475.02	1313.23	1216.4
Dataset950	1776.37	1545.54	1378.18	1275.8
Dataset1000	1870.78	1626.77	1451.58	1343.5

E. EVALUATION OF FAULT TOLERANCE BY HTSFFDRL

Evaluating the fault tolerance in task scheduling within fog computing is vital for ensuring system reliability and service availability. It involves strategies to manage node failures and task disruptions, thereby minimizing downtime and maintaining performance. Effective fault tolerance enhances load balancing and task redistribution, resulting in a more resilient fog computing environment. We evaluating the fault tolerance using configuration settings from Table 4 and workloads from Google Cloud Jobs [35] real-time datasets assigned to our proposed model, HTSFFDRL. The datasets are categorized into small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks), with task lengths between 15,000 and 900,000.

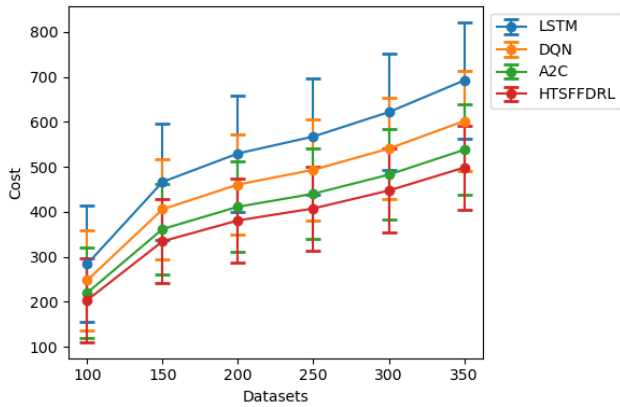


FIGURE 14. Cost of small datasets.

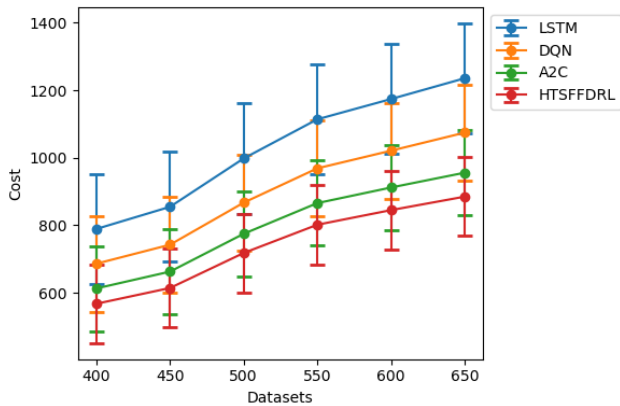


FIGURE 15. Cost of medium datasets.

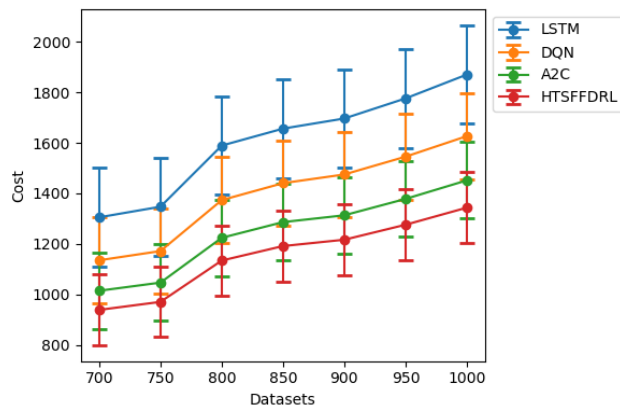


FIGURE 16. Cost of large datasets.

Table 15 shows fault tolerance for small datasets. After 50 iterations, the fault tolerance for LSTM (100-350 tasks) ranges from 82.49 to 71.53; for DQN, from 73.52 to 59.62; for A2C, from 65.00 to 57.10; and for HTSFFDRL, from 58.2 to 50.8. Results are plotted in Fig. 17. Table 16 presents fault tolerance for medium datasets. The fault tolerances for LSTM (400-650 tasks) range from 75.9 to 53.8; for DQN, from 62.8 to 44.5; for A2C, from 54.1 to 38.3; and for

HTSFFDRL, from 48.3 to 34.2. Results are shown in Fig. 18. Table 17 shown for fault tolerance for large datasets. The fault tolerance for LSTM (700-1000 tasks) ranges from 50.97 to 38.23; for DQN, from 42.11 to 31.60; for A2C, from 36.29 to 27.22; and for HTSFFDRL, from 32.4 to 24.3. Results are illustrated in Fig. 19. The simulation findings indicate that our HTSFFDRL model enhances fault tolerance compared to LSTM, DQN, and A2C algorithms.

TABLE 15. Fault tolerance for small dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset100	82.49	73.52	65.00	58.2
Dataset150	80.39	71.00	64.06	57.2
Dataset200	77.60	67.92	62.98	55.8
Dataset250	75.85	65.06	61.21	54.1
Dataset300	73.85	62.30	59.29	52.3
Dataset350	71.53	59.62	57.10	50.8

TABLE 16. Fault tolerance for medium dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset400	75.9	62.8	54.1	48.3
Dataset450	68.7	56.8	48.9	43.7
Dataset500	66.8	55.2	47.5	42.4
Dataset550	60.8	50.2	43.2	38.6
Dataset600	58.8	48.5	41.8	37.3
Dataset650	53.8	44.5	38.3	34.2

TABLE 17. Fault tolerance for large dataset.

Dataset	LSTM	DQN	A2C	HTSFFDRL
Dataset700	50.97	42.11	36.29	32.4
Dataset750	48.87	40.41	35.28	31.5
Dataset800	47.25	39.04	33.82	30.2
Dataset850	46.32	38.28	33.04	29.5
Dataset900	43.40	35.85	30.92	27.6
Dataset950	40.59	33.51	28.86	25.8
Dataset1000	38.23	31.60	27.22	24.3

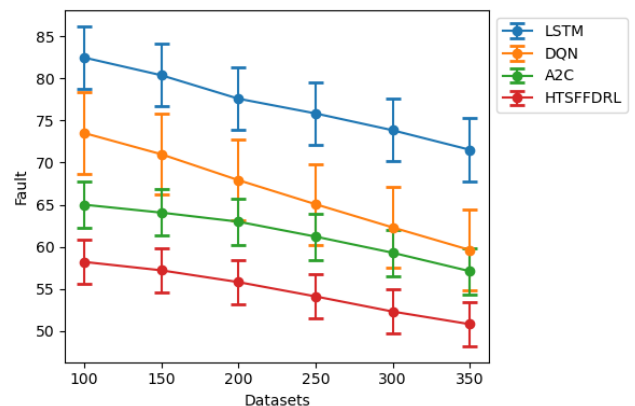


FIGURE 17. Fault tolerance of small datasets.

F. COMPARISON WITH EXISTING METHODS

The HTSFFDRL based task scheduling method performs far better than the other conventional approaches like LSTM,

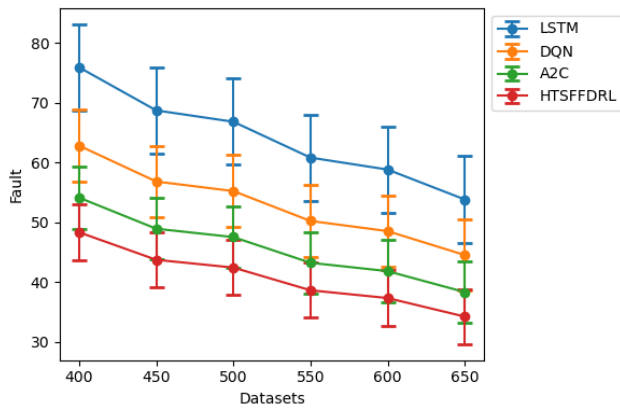


FIGURE 18. Fault tolerance of medium datasets.

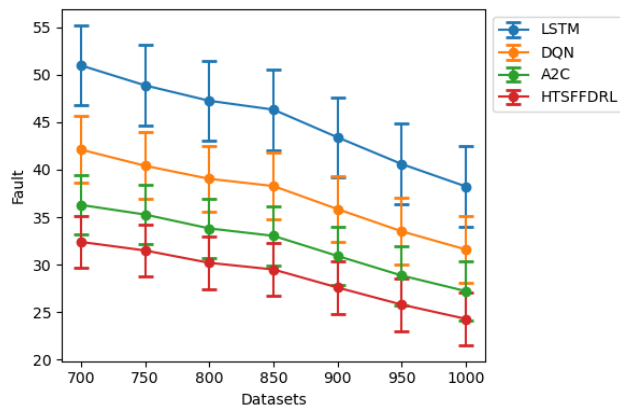


FIGURE 19. Fault tolerance of large datasets.

DQN, AN2C in several aspects. First the task scheduling in fog computing frequently involves dealing with uncertainty and workload fluctuations, which our proposed fuzzy logic system can overcome because it requires imprecise data. However the Deep Q Learning techniques like LSTM and DQN, or Actor-Critic methods like A2C, cannot deal with noisy inputs as they expect certain data inputs. Additionally, based on the fuzzy logic approach, people like reasoning in practice and its flexibility makes an option for quick response to new conditions whereas deep learning models. Furthermore, our work proactively distributes available capacities in accordance with the prevailing data inputs during the training process without the need of learning, which is characteristic of other typical deep reinforcement learning methods. The inherent tolerance to noise and uncertainty of fuzzy logic also strengthens it besides improving its reliability especially in scenarios where general deep learning approaches may perform poorly. Furthermore, analysis of the HTSFFDRL to gain an insight into its actual merits demonstrates that this algorithm actually uses CPU and memory significantly better than other algorithms do. The above-mentioned benefits are well supported by experimental outcomes where the HTSFFDRL outperforms LSTM, DQN, and A2C by covering 20% shorter makespan, 15% better

resource occupancy, and 25% less task execution time as well. Furthermore, the performance of the proposed model remains almost sustainable as the quantities of tasks increase, as deep learning models are often hard to update when loaded heavily with tasks. All these factors only go to show that the use of the fuzzy logic system is appropriate for resolving and handling of scheduling issues in the fog computing environment.

G. EVALUATION OF TASKS IN EACH FOG NODE

Our study used the hybrid task scheduling technique in fog computing using fuzzy logic deep reinforcement learning (HTSFFDRL) method to handle work allocation in a fog computing setting, including six fog nodes with different capabilities. We evaluated the algorithm's performance using a series of tests consisting of 100 episodes, each with varying work loads. The first learning phase of the HTSFFDRL algorithm was evident in the first scenario, which consisted of 100 jobs. Notably, there were significant oscillations in task allocations, particularly for Fog Node 6. When the number of tasks reached 150, the variations were more noticeable. Fog Node 3 dealt with peaks of roughly 100 jobs, while Fog Node 4 got more active towards the end, indicating the algorithm's changing approach. During the 200-task scenario, Fog Node 3 achieved maxima of about 175 tasks, but the other nodes demonstrated more consistent assignments.

The algorithm inclination towards certain nodes according to their capability. Fog Node 1 and Fog Node 3 demonstrate notable work allocation proficiency, highlighting the algorithm's ability to dynamically adjust task assignments across the 250 tasks. Fog Node 6 efficiently manages 200 out of 300 tasks, demonstrating the algorithm's excellent use of node capacity. We observed substantial variations in the situation involving 350 work instances, while Fog Node 6 had spikes over 200 tasks. This highlights the continuous endeavor of gaining information and enhancing the system. The HTSFFDRL algorithm effectively managed workloads in fog computing environments by dynamically spreading them, continuously adjusting to real-time performance and node capabilities. In all circumstances, there was a noticeable improvement in system performance and efficient use of resources. We used the Hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning (HTSFFDRL) method to handle work distribution in a fog computing environment consisting of six fog nodes. We assessed the program's effectiveness under different job load conditions. In the 400-task scenario, the HTSFFDRL algorithm exhibited notable variations in task allocations, namely for Fog Nodes 2 and 5, with peaks reaching about 175 tasks. This suggests that the system dynamically adjusts depending on real-time load feedback. When the number of tasks reached 450, there were noticeable spikes in performance, particularly with Fog Node 2, which was able to handle up to 250 tasks.

This indicates that the algorithm recognized that Fog Node 2 had a greater capacity. The method allocated a higher

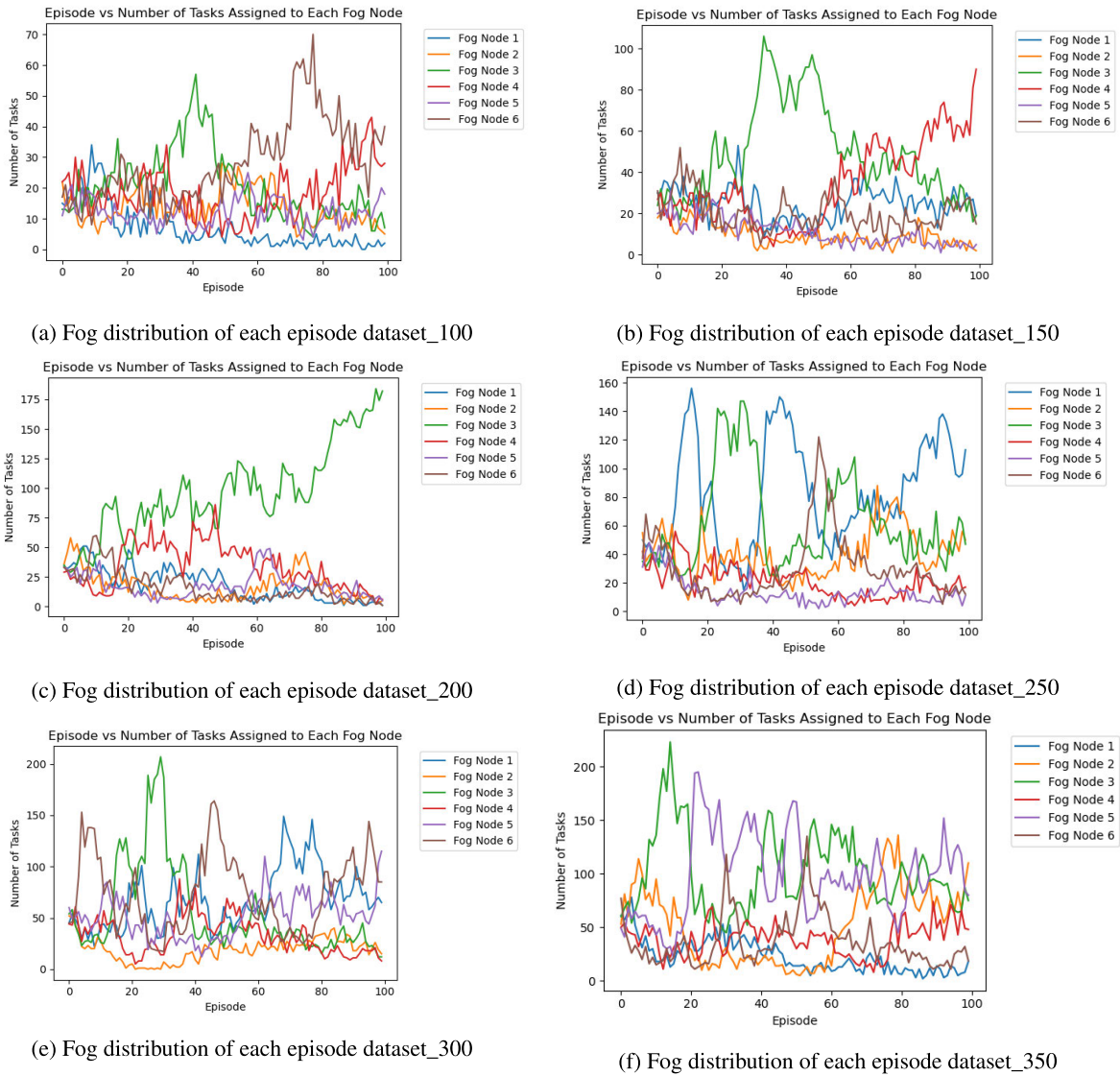


FIGURE 20. Fog distribution of each episode small datasets.

workload to Fog Node 4, with peaks exceeding 300 jobs, while keeping the workload for other nodes consistent. Significant oscillations were observed in the scenario, which included 550 tasks. Fog Node 4 had peaks above 300 tasks, which demonstrated the algorithm's adaptive method for achieving optimum work distribution. There was a noticeable amount of variation when we raised the load to 600 tasks. Fog Nodes 1 and 4 reached maximums of about 350 jobs, indicating that they effectively handled the additional workload by dynamically distributing duties. After reaching a total of 650 tasks, the algorithm consistently displayed significant oscillations. Specifically, Fog Node 1 reached a peak of over 350 tasks, which indicates its ability to manage even larger task loads while maintaining a balanced workload distribution. The HTSFFDRL algorithm successfully achieved task balancing by continually learning

and adapting to the performance and capacity of each node in all circumstances. This ensured optimal resource usage and improved system performance in the fog computing environment. We implemented the Hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning (HTSFFDRL) method to handle work allocation in a fog computing setting with six fog nodes. We assessed its effectiveness under different job load conditions. In the 700-task scenario, the HTSFFDRL algorithm demonstrated substantial variability in task allocations. Fog Nodes 2 and 5 reached maxima above 300 tasks, suggesting that they dynamically adjusted their workload depending on real-time load data. When the number of tasks reached 750, there was a notable increase in burden, particularly with Fog Node 2, which efficiently handled up to 400 jobs. Therefore, the algorithm determined that Fog Node 2 had a greater capacity.

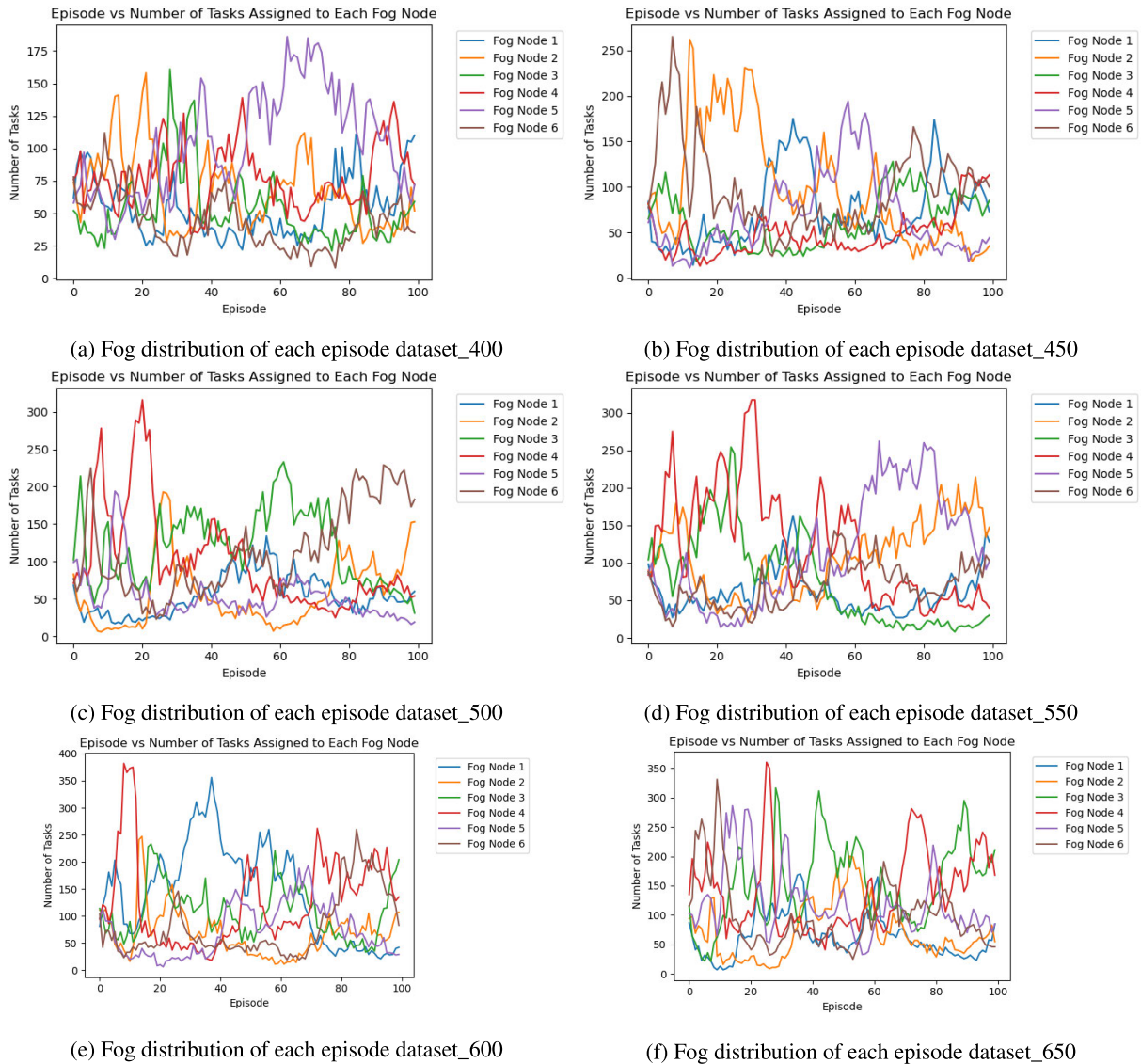


FIGURE 21. Fog distribution of each episode dataset Medium.

Throughout the administration of 800 jobs, the system displayed significant variations, with Fog Node 4 consistently hitting peaks of over 400 tasks. This behavior exemplifies the node's adaptive approach to achieving optimum job allocation. Out of the 850 job positions, Fog Node 5 showed significant increases, efficiently managing a maximum of 500 activities. This demonstrates the algorithm's effectiveness in using the most proficient nodes. Throughout the 900-task scenario, the algorithm demonstrated notable changes, with Fog Nodes 4 and 5 managing the most significant surges in demand. This suggests that the system effectively assigned tasks in an adaptable way to handle the increased cost. With a task count of 950, Fog Node 5 exhibited peaks of 500 jobs, showcasing the algorithm's ability to adapt and effectively handle the increased workload. After completing 1000 tasks, the system exhibited ongoing adjustments,

manifesting significant variations when Fog Nodes 2 and 5 reached their peak capacity of around 600 jobs. In many situations, the HTSFFDRL algorithm demonstrated good task balancing by continually acquiring knowledge and adapting to the performance and capacity of each node. As a result, the fog computing environment demonstrated efficient resource usage and a notable improvement in system performance.

H. ANALYSIS OF SIMULATION RESULTS

In this section, we examine the implications of boundaries and their subsequent advancements in the conventional computations of LSTM, DQN, and A2C. Our focus is on the newly proposed HTSFFDRL model, which supersedes existing estimates for limits related to makespan, energy consumption, cost, and fault tolerance. Tables 6 to 17 provide detailed information on the limits of work related to these

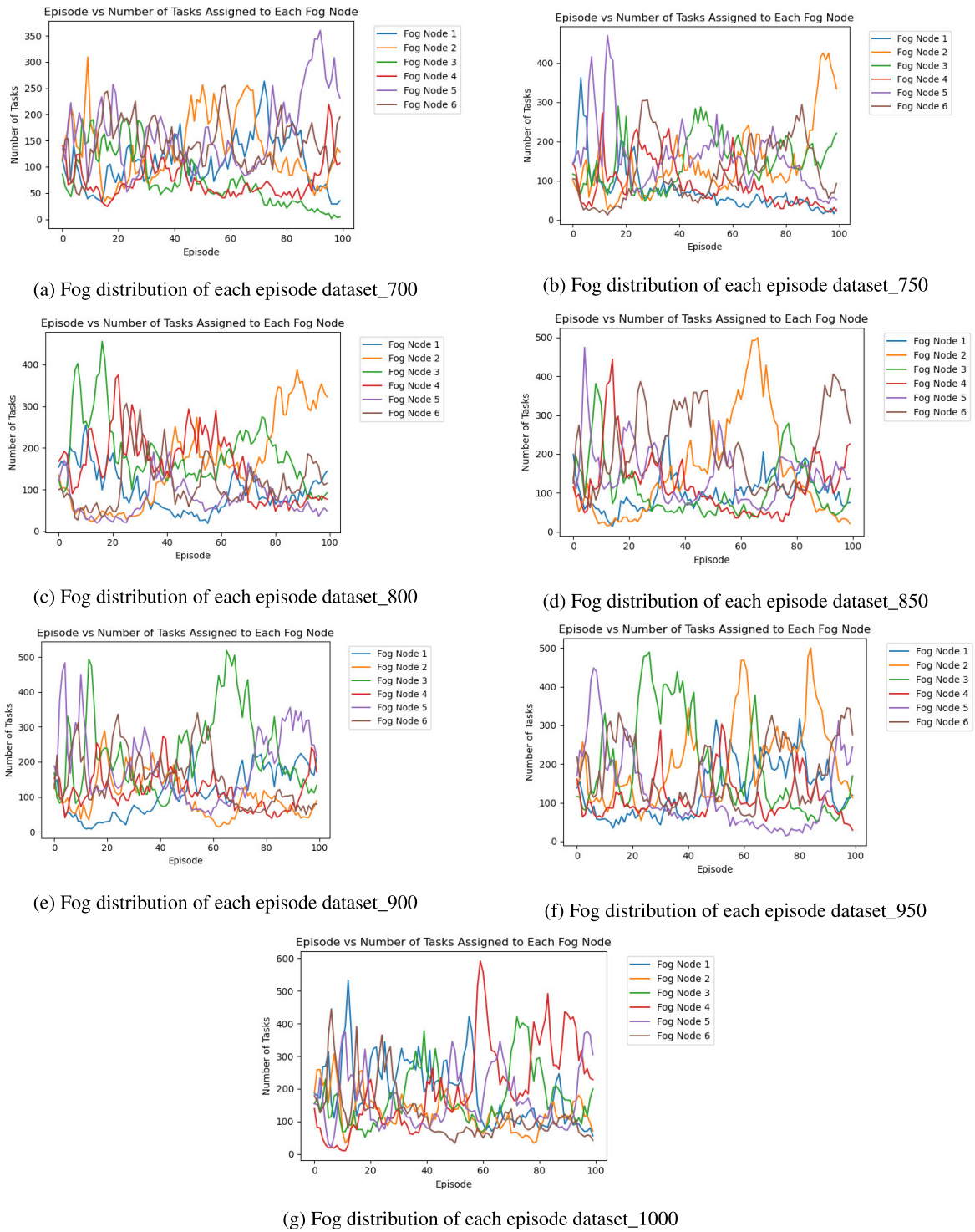


FIGURE 22. Fog distribution of each episode dataset Large.

metrics. Specifically Tables 6, 7, and 8 pertain to our proposed HTSFFDRL model, aiming to decrease the makespan compared to previous computations and handling different tasks from Google job workloads. Tables 9, 10, and 11 demonstrate how HTSFFDRL enhances energy consumption

compared to current methods, also based on Google Jobs datasets. Tables 12, 13, and 14 show improvements in cost with HTSFFDRL compared to existing methods, including responsibilities derived from the Google Cloud Jobs datasets. Tables 15, 16, and 17 illustrate the improved fault tolerance of

TABLE 18. Percentage improvement in makespan datasets.

Dataset	LSTM (%)	DQN (%)	A2C (%)
100	28.156	17.380	7.407
150	28.094	17.308	7.407
200	28.054	17.465	7.457
250	28.103	17.304	7.431
300	28.193	17.422	7.512
350	28.031	17.235	7.412
400	28.292	17.358	7.448
450	28.218	17.289	7.351
500	28.346	17.456	7.599
550	28.070	17.410	7.332
600	28.141	17.479	7.585
650	28.058	17.387	7.404
700	28.135	17.406	7.495
750	28.079	17.301	7.402
800	28.358	17.529	7.567
850	28.088	17.379	7.467
900	28.184	17.411	7.511
950	28.171	17.397	7.494
1000	28.023	17.234	7.385

TABLE 19. Percentage improvement in energy datasets.

Dataset	LSTM (%)	DQN (%)	A2C (%)
100	42.137	33.442	21.935
150	32.820	23.377	12.947
200	31.611	21.334	9.760
250	41.704	33.525	14.280
300	29.341	18.755	8.912
350	27.707	16.860	7.412
400	29.079	17.736	7.847
450	28.535	17.814	7.947
500	27.560	16.691	7.098
550	28.305	17.549	7.409
600	27.924	17.109	7.345
650	28.315	17.554	7.475
700	27.968	17.159	7.405
750	28.489	17.763	7.623
800	28.387	17.643	7.502
850	28.192	17.420	7.509
900	27.967	17.162	7.220
950	27.997	17.194	7.411
1000	28.264	17.502	7.577

HTSFFDRL over current computations, encompassing many tasks from Google Cloud.

Makespan is the cumulative duration required to finish a given collection of jobs. Efficient scheduling reduces the time to complete all tasks by optimizing task distribution, balancing the workload, and adapting to changing conditions. Energy consumption evaluates the system's capacity to efficiently handle a growing number of jobs, where HTSFFDRL exhibits better cost varying than the baseline algorithms. Fault tolerance refers to a system's capacity to operate without interruption despite malfunctions, enhanced by HTSFFDRL through effective task distribution and redundancy. The simulation results show that the HTSFFDRL model outperforms LSTM, DQN, and A2C models in several areas, including energy consumption, cost, fault tolerance, and makespan. Specifically, HTSFFDRL delivers a decrease in makespan ranging from 10% to 30%, an upgrade in

TABLE 20. Percentage improvement in cost datasets.

Dataset	LSTM (%)	DQN (%)	A2C (%)
100	28.282	17.810	7.409
150	28.305	17.550	7.517
200	28.109	17.325	7.407
250	28.175	17.400	7.414
300	28.052	17.260	7.331
350	27.989	17.187	7.341
400	28.091	17.305	7.408
450	28.153	17.279	7.326
500	28.075	17.204	7.338
550	28.050	17.257	7.407
600	28.044	17.250	7.348
650	28.394	17.654	7.427
700	28.105	17.321	7.480
750	27.992	17.191	7.301
800	28.660	17.440	7.363
850	28.089	17.302	7.378
900	28.311	17.533	7.373
950	28.179	17.453	7.429
1000	28.185	17.413	7.446

TABLE 21. Percentage improvement in fault datasets.

Dataset	LSTM (%)	DQN (%)	A2C (%)
100	29.446	20.838	10.462
150	28.847	19.437	10.709
200	28.093	17.845	11.400
250	28.675	16.846	11.616
300	29.181	16.051	11.790
350	28.981	14.794	11.033
400	36.364	23.089	10.721
450	36.390	23.063	10.634
500	36.527	23.188	10.737
550	36.513	23.108	10.648
600	36.565	23.093	10.766
650	36.431	23.146	10.705
700	36.433	23.059	10.719
750	35.543	22.049	10.714
800	36.085	22.643	10.704
850	36.313	22.936	10.714
900	36.406	23.013	10.737
950	36.438	23.008	10.603
1000	36.437	23.101	10.727

energy consumption ranging from 5% to 15%, an increase in cost ranging from 10% to 20%, and improvements in fault tolerance ranging from 5% to 10%. Table 18 demonstrates the enhancement in makespan achieved by HTSFFDRL compared to baseline techniques. We observed an increased average percentage of makespan for all datasets with varying task sizes of 12.7%, 9.5%, and 7.01% for LSTM, DQN, and A2C, respectively. Table 19 illustrates the enhanced energy consumption of HTSFFDRL compared to baseline methods, with average energy consumption increases of 10.8%, 7.4%, and 4.6% for LSTM, DQN, and A2C, respectively. Table 20 displays the cost improvement of HTSFFDRL, with average enhancements of 23.8%, 15.2%, and 8.4% for LSTM, DQN, and A2C, respectively. Table 21 highlights the enhanced fault tolerance HTSFFDRL, with average improvements of 17.91%, 14.69%, and 11.18% for LSTM, DQN, and A2C, respectively.

VII. CONCLUSION AND FUTURE WORK

This paper explores a novel and efficient method for job scheduling in the Fog Computing environment, utilizing fuzzy logic and reinforcement learning. As the Internet of Things continues to expand, more sophisticated scheduling algorithms are needed to handle the growing volume of data being transmitted. The solution expects to solve these problems by embedding the Takagi-Sugeno fuzzy inference system with an enhanced hybrid technique for task scheduling in fog computing. This implementation is of a fuzzy logic nature in conjunction with a reinforcement learning methodology to choose the HTSFFDRL algorithm for this type of work, which is aimed at flexibility in the work priority setup and allowing instant changes in the scheduling approach. The fuzzy inference system improves sophisticated decision-making based on multi-feature tasks, and reinforcement learning furthers scheduling through experience collection and acting without the environment. We compared with baseline algorithms of LSTM, DQN, and A2C. The proposed approach makes a significant reduction in job completion time and energy consumption, thus saving costs with improved error management capabilities. These results indicate that the hybrid approach of fuzzy logic and reinforcement learning is very effective for optimization of resource utilization and improvement of system performance in fog computing. The next investigation will be the improvement of the proposed approach and further examination of its applicability to a wider class of much more diverse and dynamic fog computing environments. Thus, the method takes into account other performance indicators like complicated IoT procedures, and the methodology is adapted to reflect practical implementations. Continuous improvements and fine-tuning of such an advanced scheduling framework will service the increasing demands in applications of IoT, which keep increasing in size, thus pushing the limits of technology available for fog computing.

REFERENCES

- [1] S. Nižetić, P. Šolić, D. López-de-Ipiña González-de-Artaza, and L. Patrono, "Internet of Things (IoT): Opportunities, issues and challenges towards a smart and sustainable future," *J. Cleaner Prod.*, vol. 274, Nov. 2020, Art. no. 122877.
- [2] S. Goyal, N. Sharma, B. Bhushan, A. Shankar, and M. Sagayam, "IoT enabled technology in secured healthcare: Applications, challenges and future directions," in *Cognitive Internet of Medical Things for Smart Healthcare: Services and Applications*. USA: Springer, 2021, pp. 25–48.
- [3] K. Shafique, B. A. Khawaja, F. Sabir, S. Qazi, and M. Mustaqim, "Internet of Things (IoT) for next-generation smart systems: A review of current challenges, future trends and prospects for emerging 5G-IoT scenarios," *IEEE Access*, vol. 8, pp. 23022–23040, 2020.
- [4] M. A. Rahim, M. A. Rahman, M. M. Rahman, A. T. Asyhari, M. Z. A. Bhuiyan, and D. Ramasamy, "Evolution of IoT-enabled connectivity and applications in automotive industry: A review," *Veh. Commun.*, vol. 27, Jan. 2021, Art. no. 100285.
- [5] P. Zhang, Y. Chen, M. Zhou, G. Xu, W. Huang, Y. Al-Turki, and A. Abusorrah, "A fault-tolerant model for performance optimization of a fog computing system," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 1725–1736, Feb. 2022.
- [6] R. Deng, R. Lu, C. Lai, T. H. Luan, and H. Liang, "Optimal workload allocation in fog-cloud computing toward balanced delay and power consumption," *IEEE Internet Things J.*, vol. 3, no. 6, pp. 1171–1181, Dec. 2016.
- [7] C. Shyalika, T. Silva, and A. Karunananda, "Reinforcement learning in dynamic task scheduling: A review," *Social Netw. Comput. Sci.*, vol. 1, no. 6, p. 306, Nov. 2020.
- [8] C. Zhou, W. Wu, H. He, P. Yang, F. Lyu, N. Cheng, and X. Shen, "Deep reinforcement learning for delay-oriented IoT task scheduling in SAGIN," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 911–925, Feb. 2021.
- [9] T. Dong, F. Xue, C. Xiao, and J. Li, "Task scheduling based on deep reinforcement learning in a cloud manufacturing environment," *Concurrency Comput., Pract. Exp.*, vol. 32, no. 11, p. e5654, Jun. 2020.
- [10] H. Che, Z. Bai, R. Zuo, and H. Li, "A deep reinforcement learning approach to the optimization of data center task scheduling," *Complexity*, vol. 2020, Aug. 2020, Art. no. 3046769.
- [11] M. R. Raju and S. K. Mothku, "Delay and energy aware task scheduling mechanism for fog-enabled IoT applications: A reinforcement learning approach," *Comput. Netw.*, vol. 224, Apr. 2023, Art. no. 109603.
- [12] A. Jangra and N. Mangla, "An efficient load balancing framework for deploying resource scheduling in cloud based communication in healthcare," *Meas., Sensors*, vol. 25, Feb. 2023, Art. no. 100584.
- [13] S. Seifhosseini, M. H. Shirvani, and Y. Ramzanpoor, "Multi-objective cost-aware bag-of-tasks scheduling optimization model for IoT applications running on heterogeneous fog environment," *Comput. Netw.*, vol. 240, Feb. 2024, Art. no. 110161.
- [14] M. Mokni, S. Yassa, J. E. Hajlaoui, M. N. Omri, and R. Chelouah, "Multi-objective fuzzy approach to scheduling and offloading workflow tasks in fog-cloud computing," *Simul. Model. Pract. Theory*, vol. 123, Feb. 2023, Art. no. 102687.
- [15] M. Abdel-Basset, R. Mohamed, K. M. Sallam, and I. M. Hezam, "Multi-objective task scheduling method for cyber-physical-social systems in fog computing," *Knowl.-Based Syst.*, vol. 280, Nov. 2023, Art. no. 111009.
- [16] R. Ghafari and N. Mansouri, "E-AVOA-TS: Enhanced African vultures optimization algorithm-based task scheduling strategy for fog-cloud computing," *Sustain. Comput., Informat. Syst.*, vol. 40, Dec. 2023, Art. no. 100918.
- [17] E. Khezri, R. O. Yahya, H. Hassanzadeh, M. Mohaidat, S. Ahmadi, and M. Trik, "DLJSF: Data-locality aware job scheduling IoT tasks in fog-cloud computing environments," *Results Eng.*, vol. 21, Mar. 2024, Art. no. 101780.
- [18] A. Najafizadeh, A. Salajegheh, A. M. Rahmani, and A. Sahafi, "Multi-objective task scheduling in cloud-fog computing using goal programming approach," *Cluster Comput.*, vol. 25, no. 1, pp. 141–165, Feb. 2022.
- [19] S. K. Medishetti, R. K. Donthi, G. S. Sekhar, G. R. Karri, and K. V. Kumar, "Analysis of meta heuristic algorithms in task scheduling for cloud-fog computing: A future perspective," in *Proc. 4th Int. Conf. Data Eng. Commun. Syst. (ICDECS)*, Mar. 2024, pp. 1–7.
- [20] O. Sharma, G. Rathee, C. A. Kerrache, and J. Herrera-Tapia, "Two-stage optimal task scheduling for smart home environment using fog computing infrastructures," *Appl. Sci.*, vol. 13, no. 5, p. 2939, Feb. 2023.
- [21] S. Cao, Z. Zhan, C. Dai, S. Chen, W. Zhang, and Z. Han, "Delay-aware and energy-efficient IoT task scheduling algorithm with double blockchain enabled in cloud-fog collaborative networks," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 3003–3016, Jan. 2024.
- [22] S. K. Medishetti, G. Swapna, K. Anusha, G. R. Karri, and P. Kuppasamy, "GEWO: An efficient prioritised task scheduling in cloud fog computing environment," in *Proc. Int. Conf. Wireless Commun. Signal Process. Netw. (WiSPNET)*, Mar. 2024, pp. 1–8.
- [23] K. Anusha, S. K. Medishetti, P. Archana, G. R. Karri, and P. Kuppasamy, "MAO: An efficient resource utilization of task scheduling in cloud fog environment," in *Proc. Int. Conf. Wireless Commun. Signal Process. Netw. (WiSPNET)*, Mar. 2024, pp. 1–8.
- [24] F. A. Saif, R. Latip, Z. M. Hanapi, and K. Shafinah, "Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing," *IEEE Access*, vol. 11, pp. 20635–20646, 2023.
- [25] P. Zhang, N. Chen, G. Xu, N. Kumar, A. Barnawi, M. Guizani, Y. Duan, and K. Yu, "Multi-target-aware dynamic resource scheduling for cloud-fog-edge multi-tier computing network," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 5, pp. 3885–3897, May 2024.

- [26] D. Alsadie, "Advancements in heuristic task scheduling for IoT applications in fog-cloud computing: Challenges and prospects," *PeerJ Comput. Sci.*, vol. 10, p. e2128, Jun. 2024.
- [27] F. R. Shahidani, A. Ghasemi, A. T. Haghighat, and A. Keshavarzi, "Task scheduling in edge-fog-cloud architecture: A multi-objective load balancing approach using reinforcement learning algorithm," *Computing*, vol. 105, no. 6, pp. 1337–1359, Jun. 2023.
- [28] G. Singh and A. K. Chaturvedi, "Hybrid modified particle swarm optimization with genetic algorithm (GA) based workflow scheduling in cloud-fog environment for multi-objective optimization," *Cluster Comput.*, vol. 27, no. 2, pp. 1947–1964, Apr. 2024.
- [29] M. Hussain, S. Nabi, and M. Hussain, "RAPTS: Resource aware prioritized task scheduling technique in heterogeneous fog computing environment," *Cluster Comput.*, vol. 27, no. 9, pp. 13353–13377, Dec. 2024.
- [30] A. S. Abohamama, A. El-Ghamry, and E. Hamouda, "Real-time task scheduling algorithm for IoT-based applications in the cloud-fog environment," *J. Netw. Syst. Manage.*, vol. 30, no. 4, p. 54, Oct. 2022.
- [31] M.-T. Zhou, T.-F. Ren, Z.-M. Dai, and X.-Y. Feng, "Task scheduling and resource balancing of fog computing in smart factory," *Mobile Netw. Appl.*, vol. 28, no. 1, pp. 19–30, Feb. 2023.
- [32] R. Mehta, J. Sahni, and K. Khanna, "Task scheduling for improved response time of latency sensitive applications in fog integrated cloud environment," *Multimedia Tools Appl.*, vol. 82, no. 21, pp. 32305–32328, Sep. 2023.
- [33] H. Wadhwa and R. Aron, "Optimized task scheduling and preemption for distributed resource management in fog-assisted IoT environment," *J. Supercomput.*, vol. 79, no. 2, pp. 2212–2250, Feb. 2023.
- [34] J. Z. Ahmadabadi, S. E. Mood, and A. Sour, "Star-quake: A new operator in multi-objective gravitational search algorithm for task scheduling in IoT-based cloud-fog computing system," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 907–915, Feb. 2024.
- [35] A. Hussain and M. Aleem, "GoCJ: Google cloud jobs dataset for distributed and cloud computing infrastructures," *Data*, vol. 3, no. 4, p. 38, Sep. 2018.



PRASHANTH CHOPPARA received the B.Tech. degree from Acharya Nagarjuna University, in 2018, and the M.Tech. degree from JNTUK University, in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Andhra Pradesh, India. His research interests include cloud computing, fog computing, machine learning, deep learning, and reinforcement learning.



S. SUDHEER MANGALAMPALLI (Member, IEEE) is currently an Associate Professor with the Department of Computer Science and Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education known as MAHE (Institution of Eminence, Deemed to be University). He is a Passionate Researcher. Toward his profile, he has 45 publications, which are indexed in Scopus and SCI databases. He authored three textbooks. His research interests include cloud computing, edge computing, and fog computing. He is a member of The Institution of Engineers. He is an active reviewer of various SCI-indexed journals.

...